



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institut für Integrierte Systeme
Integrated Systems Laboratory

DEPARTMENT OF INFORMATION TECHNOLOGY AND
ELECTRICAL ENGINEERING

Spring Semester 2025

WI-Body: towards an ultra-low power and high throughput network for continuous body monitoring

Bachelor Thesis

Alberto Fasulo
afasulo@student.ethz.ch

may 2025

Supervisors: Sebastian Frey, sebastian.frey@iis.ee.ethz.ch
Giusy Spacone, gspacone@student.ethz.ch
Dr. Andrea Cossettini, cossettini.andrea@iis.ee.ethz.ch
Dr. Victor Kartsch, victor.kartsch@iis.ee.ethz.ch
Professor: Prof. Dr. Luca Benini, lbenini@ethz.ch

Acknowledgements

I would like to express my deepest gratitude to all those who supported and guided me throughout the course of this thesis.

First and foremost, I am sincerely grateful to my supervisors, Sebastian Frey and Giusy Spacone, for their guidance and availability during the course of my thesis.

I am also grateful to Dr. Andrea Cossettini, and Dr. Victor Kartsch, for keeping an eye on me during the entire work.

Another thanks goes to Professor Dr. Luca Benini for giving me the opportunity to work on this project.

And finally i want to express my gratitude to all my friends and family who supported me during the totality of this thesis.

Abstract

Wireless Body Area Networks (WBANs) are becoming essential for continuous health monitoring, wearable biomedical sensing, and real-time physiological data acquisition. These systems require hardware platforms that combine high data throughput with low power consumption to enable long-term, unobtrusive operation on the human body. Conventional WBAN solutions based on Bluetooth Low Energy (BLE) or IEEE 802.15.4 typically offer excellent energy efficiency but suffer from limited bandwidth and scalability.

This thesis investigates the use of the ESP32-C6, a modern Wi-Fi 6-enabled System-on-Chip (SoC), as a candidate platform for high-throughput, energy-aware WBAN applications. The ESP32-C6 integrates advanced low-power modes and supports Target Wake Time (TWT), a key Wi-Fi 6 feature that allows scheduled communication and reduced idle listening.

A custom WBAN testbed was developed to evaluate the platform under realistic biosignal transmission workloads. Experimental results show that the ESP32-C6 is capable of sustaining data rates in the multi-megabit-per-second range. Power consumption was significantly reduced during idle periods using modem-sleep and light-sleep modes. In addition, precise inter-node synchronization within $\pm 50 \mu\text{s}$ was achieved using a TSF-based timing protocol.

These results demonstrate that Wi-Fi 6 platforms such as the ESP32-C6 can effectively meet the demands of next-generation WBANs, offering a promising balance of throughput, timing precision, and energy efficiency for wearable health technologies.

Declaration of Originality

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor. For a detailed version of the declaration of originality, please refer to Appendix B

Alberto Fasulo,
Zurich, may 2025

Contents

Acknowledgements	ii
List of Acronyms	x
1. Introduction	1
1.1. Wireless Body Area Networks (WBAN)	1
1.2. Current State of the Art	3
1.3. Scope and Objectives	4
2. Preliminaries	5
2.1. BioGAP Platform and WiFi Shield	5
3. Theory / Algorithms	7
3.1. Power Optimisation	7
3.1.1. ESP-only network	8
3.1.2. Non ESP-only Network	11
3.2. Time Synchronization	15
3.2.1. IEEE 802.11's Built-in Timing Synchronization Function	15
4. Firmware Architecture	17
4.1. System Architecture	17
4.1.1. System Characterization	17
4.1.2. Firmware Structure	17
4.1.3. All-ESP Network	18
4.1.4. ESP-to-Router	20
5. Design Implementation	21
5.1. Power Optimization	21
5.1.1. Current Profile Analysis in Light Sleep Mode	22
5.1.2. All-ESP	23
5.1.3. Non All-ESP Network	25

Contents

5.1.4. Time Synchronization Function	28
5.1.5. Packet Header	28
6. Results	32
6.1. Power Optimization	32
6.1.1. All-ESP	32
6.1.2. Non-All-ESP	34
6.2. TSF	38
6.2.1. TSF with Two Clients	38
6.2.2. TSF: Two Clients and AP	40
6.2.3. Beacon Interval	41
6.3. Extra	42
7. Conclusion and Future Work	45
7.1. Power Optimization	45
7.2. Time Synchronization	46
7.3. Summary and Future Directions	46
A. Task Description	48
B. Declaration of Originality	54

List of Figures

1.1. Typical WBAN application fields, types, and examples. Adapted from [1].	2
3.1. Functional block diagram of the ESP32-C6 SoC (adapted from Espressif's product announcement[2]).	8
3.2. TWT Parameter Set and Request Type field formats. The TWT Wake Interval Mantissa is a 2-byte (16-bit) field, and the TWT Wake Interval Exponent is a 5-bit field. Together, they define the interval between scheduled wake-ups.	13
4.1. High-level architecture of the Access Point firmware (developed by a previous student).	18
4.2. High-level architecture of the client firmware.	19
5.1. Time based cycle flow diagram	22
5.2. Example of current consumption.	23
5.3. Structure of the custom packet header used for time-synchronized data collection.	29
5.4. Experimental setup for time synchronization: both ESP32-C6 clients and the AP receive the trigger from the waveform generator. Image assembled in PowerPoint using elements from Keysight E3647A [3], Keysight 33210A [4], ESP32-C6 [5], and breadboard [6].	31
6.1. Results for the 100ms total period with different duty cycle, both for AP and client.	33
6.2. Results for the 1s total period with different duty cycle, both for AP and client.	33
6.3. Results for the 100s total period with different duty cycle, both for AP and client.	34
6.4. Results for the 100 ms total period with different duty cycles for the client.	35
6.5. Results for the 250 ms total period with different duty cycles for the client.	36

List of Figures

6.6. Results for the 1 s total period with different duty cycles for the client. . .	36
6.7. Results for the 100 s total period with different duty cycles for the client. .	37
6.8. Box plot of synchronization error between two clients, illustrating the distribution, variance, and precision of the timing alignment over the experiment duration.	39
6.9. Synchronization error between two clients as a function of time over a 40-minute period, used to assess long-term stability and potential drift in synchronization performance.	39
6.10. Synchronization error between two clients and the AP over time, illustrating the TSF alignment across all nodes in the network.	40
6.11. Box plot of the absolute time difference between client and AP for various beacon intervals.	41
6.12. Power consumption as a function of beacon interval. The mean synchronization error for each interval is annotated.	42
6.13. Throughput comparison for two client boards under varying duty cycles. .	43
6.14. Throughput degradation over increasing distance between client and access point.	44

List of Tables

1.1. Data rate, range, and power demand of WBAN-class radios (adapted from [7]).	3
3.1. Power Modes and Characteristics	11
3.2. Power-saving levers evaluated on the ESP32-C6 under two network set-ups	15

List of Acronyms

AP		Access Point
BLE		Bluetooth Low Energy
ECG		Electrocardiogram
EEG		Electroencephalogram
EMG		Electromyography
ESP		Espressif Systems Platform
IEEE		Institute of Electrical and Electronics Engineers
MAC		Media Access Control
PMU		Power Management Unit
RISC-V		Reduced Instruction Set Computer – V (open standard ISA)
RTC		Real-Time Clock
SoC		System on Chip
TCP		Transmission Control Protocol
TSF		Timing Synchronization Function
TWT		Target Wake Time
UDP		User Datagram Protocol

List of Acronyms

WBAN		Wireless Body Area Network
Wi-Fi		Wireless Fidelity

Introduction

1.1. Wireless Body Area Networks (WBAN)

A Wireless Body Area Network (WBAN) is a wireless network composed of multiple sensor-equipped nodes that are placed on, around, or even inside the human body. These nodes are designed to continuously monitor a wide array of physiological and environmental parameters, such as electrocardiogram (ECG), electroencephalogram (EEG), electromyography (EMG), blood glucose levels, body temperature, motion, and more. The data collected by these sensors are typically transmitted wirelessly to a central node, such as a dedicated receiver, a smartphone, a router, or similar devices, where it can be processed or transmitted for further analysis.

WBANs are useful in many different areas. In healthcare, they help simplify and improve patient monitoring by collecting real-time data from wearable and implantable devices, which helps doctors with the diagnosis, treatment, and early detection of health problems. Beyond clinical settings, WBANs are also used for motion tracking and secure authentication, demonstrating their versatility beyond purely medical uses.

Their utility extends to other sectors as well; WBANs are used in the military to monitor soldiers' health, spot fatigue, and warn about dangers like fires or toxic gases. In sports, they provide athletes and coaches real-time information on performance and physical condition, helping to improve training and safety. The entertainment industry uses WBANs for things like gaming, virtual reality, and personal tracking, making experiences more interactive and engaging.

The following figure, adapted from [1], provides a comprehensive overview of the main fields, types, and concrete examples of WBAN application, showing the versatility of WBAN.

1. Introduction

WBAN fields	Applications types		Examples of applications
Healthcare	Medical	Wearable	Electrocardiogram (ECG), electroencephalogram (EEG), Electromyography (EMG), Saturation of Peripheral Oxygen (SPO2), temperature, blood pressure, drugs delivery
		Implant	Diabetes control
	Non-medical	Motion detection	
		Secure authentication	
Military and defense	Medical wearable	Asses soldier fatigue, detect life threatening situations	
	Non-medical wearable	Fire detection, poisonous gas	
Sports	Medical wearable	heartbeat, temperature, blood pressure, motion sensor	
Entertainment	Non-medical wearable	Real time streaming: Video streaming by camera, audio streaming by headsets	
		Consumer electronics: MP3 player, microphone, camera	
		Gaming purposes, virtual reality, ambient, intelligence areas, personal item tracking and social networking	

Figure 1.1.: Typical WBAN application fields, types, and examples. Adapted from [1].

As wireless body area networks (WBANs) advance, their sensor nodes must deliver continuous, high-bandwidth data streams for several days while running on a single coin-cell battery. Meeting this goal requires a careful balance between two competing objectives: maximizing wireless throughput and minimizing power consumption.

Low-power radio technologies such as Bluetooth Low Energy (BLE) and IEEE 802.15.4 (ZigBee) meet the energy requirements of WBANs, consuming only a few milliwatts. However, they are significantly limited in data rate, with IEEE 802.15.4 peaking at 250 kbps and BLE reaching up to 2 Mbps in its latest iterations. In contrast, Wi-Fi (IEEE 802.11) supports data rates of hundreds of megabits per second and can leverage existing network infrastructure, but its substantially higher power draw renders it impractical for battery-powered nodes.

Table 1.1 provides an overview of the characteristics of various wireless communication standards relevant to WBANs. It serves to illustrate the typical trade-offs between throughput, range, and power consumption encountered when selecting a wireless protocol for such systems.

1. Introduction

Technology	Peak data rate	Range	Power efficiency
Bluetooth Classic	3 Mbps	10–100 m	~100 mW
Bluetooth Low Energy (BLE)	1 Mbps	10 m	~10 mW
ZigBee (802.15.4)	20–250 kbps	100 m	~30 mW
Wi-Fi (802.11 n/ac)	600 Mbps	100 m	Very High
LoRaWAN	50 kbps	2–20 km	Up to 10 years battery
SigFox	100 bps	10–50 km	10 μ W–100 mW
Ingenu	230–250 kbps	4.6 km (urban)	Up to 15 years battery
NB-IoT	200 kbps	1–10 km	200–700 mW
Wi-Fi HaLow (802.11 ah)	up to 300 Mbps	Up to 1 km	Up to 10 years battery

Table 1.1.: Data rate, range, and power demand of WBAN-class radios (adapted from [7]).

1.2. Current State of the Art

Recent literature on Wireless Body Area Networks (WBANs) demonstrates a broad focus on low-power wireless technologies and their application in different health monitoring scenarios. Bajcsy et al. [8] investigated the use of ZigBee (IEEE 802.15.4) for the collection of multichannel gyroscope data in cycling, reporting reliable operation at 250 Hz but noting packet loss rates up to 8% at higher data rates. Pantelopoulos and Bourbakis [9] provided a comprehensive survey of wearable sensor-based systems, highlighting ZigBee’s ability to support continuous ECG and motion monitoring at 100 Hz with low power consumption (~ 20 mA) and throughput up to 250 kbps, while maintaining packet loss below 1%.

Alomainy et al. [10] and Chen et al. [11] examined ZigBee-based WBANs for patient monitoring and real-time EEG acquisition, achieving reliable data transmission at sampling rates up to 500 Hz and throughputs between 15 and 32 kbps, with transmission power as low as 36.5 mW. Liu et al. [12] and Yu et al. [13] further explored ZigBee’s scalability and energy efficiency in mobile healthcare systems, confirming its suitability for multi-node, low-data-rate environments.

Bluetooth technologies have also been extensively evaluated. Xu et al. [14] demonstrated that Bluetooth Classic can support multi-sensor WBANs, achieving data rates up to 2 Mbps at 3.3 V for ECG, body temperature, and breathing rate monitoring. Touati et al. [15] experimentally validated BLE for remote ECG monitoring, showing that BLE platforms can deliver data rates up to 1 Mbps and end-to-end latencies of 4.8–7.6 ms, with stable performance on coin cell batteries (27 mA).

Recent studies have expanded the focus to Wi-Fi-based WBANs. Cavalcanti et al. [16] analyzed the performance of Wi-Fi 6 for healthcare applications, demonstrating that it can provide robust, low-latency connections and high throughput suitable for continuous,

1. Introduction

real-time health monitoring. Smith et al. [17] reviewed Wi-Fi's role in future WBANs, emphasizing its potential for high-throughput environments and showing that increased power consumption can limit its practicality for long-term wearable deployments.

In summary, these studies collectively show that ZigBee and BLE remain the leading solutions for energy-efficient WBANs, each excelling in different network and application contexts, while Wi-Fi is increasingly explored for data-intensive scenarios where power constraints are less critical. Ongoing research continues to address challenges related to scalability, packet loss, and power management to enable robust, long-term operation in real-world health monitoring systems.

1.3. Scope and Objectives

This thesis investigates whether a modern ESP32-C6¹ can achieve Wi-Fi-level throughput while meeting the strict energy constraints typical of WBAN devices. The ESP32-C6 stands out for its advanced power management, offering multiple low-power modes and dynamic subsystem control.

The study focuses on three key power-saving features available on the ESP32-C6 and supported by Wi-Fi 6:

- **Modem-sleep:** Disables the RF and baseband circuits, keeping the digital core active and reducing the current draw to a few milliamperes. This mode is ideal for conserving energy between transmissions while maintaining network connectivity.
- **Light-sleep:** Suspends the entire SoC except for the RTC domain, lowering the standby current below 1 mA. The device can quickly resume operation, preserving its state for a prompt response to events.
- **Target Wake Time (TWT):** A Wi-Fi 6 feature that schedules precise wake-up intervals with the access point, allowing the device to maximize its sleep capabilities during idle periods and significantly reduce energy consumption

These techniques are combined with a Time Synchronisation Function (TSF) to maintain precise synchronization among nodes. The primary goals are to minimize energy consumption and to keep node synchronization within $\pm 50, \mu\text{s}$, creating a reliable and synchronized WBAN.

¹A dual-band Wi-Fi 6 / IEEE 802.15.4 SoC with a RISC-V core.

Preliminaries

2.1. BioGAP Platform and WiFi Shield

The foundation of this thesis is the WiFi Shield, developed as part of a previous Bachelor thesis at ETH Zurich [18]. The WiFi Shield was designed as an expansion board for the BioGAP platform—a cutting-edge Wireless Body Area Network (WBAN) device created through collaboration between ETH Zurich and academic and industrial partners.

The **BioGAP platform** [19] features a high-performance system-on-chip (SoC) and Bluetooth Low Energy (BLE) connectivity, enabling the acquisition and processing of diverse biosignals from multiple sensor nodes distributed across the human body. This multi-sensor approach allows for comprehensive, continuous monitoring of physiological states, supporting a wide range of applications in medical diagnostics, rehabilitation, and performance analysis. However, while BLE offers low power consumption, its limited data rate constrains high-throughput applications, making it insufficient for scenarios requiring continuous, high-bandwidth data streaming.

To overcome this limitation, the **WiFi Shield** was introduced. It integrates an Espressif ESP32-C6-MINI-1-H8 microcontroller, which brings Wi-Fi 6 connectivity to the BioGAP ecosystem. This upgrade significantly increases the achievable data rates and enables robust board-to-board communication between the BioGAP and additional expansion modules. The firmware for the WiFi Shield is responsible for establishing a Wi-Fi network, managing TCP data transmission, and ensuring precise time synchronization via UDP. The shield also supports features such as microSD storage, real-time clock (RTC) integration, and voltage level shifting to ensure compatibility with various components.

Development and Prototyping: The initial development and firmware prototyping were carried out using the ESP32-C6-DevKitM-1 development board. This devkit

2. Preliminaries

provides a ready-to-use embedded platform with the Espressif microcontroller, general-purpose input/output (GPIO) pins, LEDs, and USB connectivity for easy flashing and debugging. The devkit enabled rapid firmware iteration and validation before transitioning to the custom WiFi Shield hardware.

Benchmarking and Results: Comprehensive testing of the WiFi Shield demonstrated its capabilities:

- Achieved data rates up to 14.33 Mbit/s
- Average latencies as low as 4.19 ms for data transmission
- 1.7 ms for time synchronization in single-station scenarios

These results highlight the shield's suitability for demanding WBAN applications that require both high-throughput and low-latency wireless communication.

Current Challenges: Despite these promising results, two main challenges remain:

- **Power Efficiency:** Maintaining high data rates while significantly reducing power consumption is crucial for wearable and body area network applications, where energy resources are limited.
- **Time Synchronization:** Further improving time synchronization is essential, as a latency of around 2 ms is too high for fast and responsive WBAN scenarios, such as real-time closed-loop applications and multi-sensor coordination.

The first step of this thesis was to bring up the WiFi Shield and validate its functionality through comprehensive testing, ensuring a reliable and robust foundation for further development toward ultra-low-power, high-throughput WBAN systems. Subsequent work focuses on optimizing both power consumption and synchronization precision, aiming to enable new type for wearable applications that demand both efficiency and responsiveness.

Chapter 3

Theory / Algorithms

3.1. Power Optimisation

Before discussing sleep modes, it is useful to recall which internal blocks of the ESP32-C6 are active during normal operation.

Figure 3.1 highlights six functional blocks:

1. **Core System** – A single-core 32-bit *RISC-V* CPU with cache, SRAM, ROM, and JTAG interface.
2. **WLAN** – Wi-Fi 6 MAC/baseband plus Bluetooth 5 (LE) controller, backed by the **RF** front-end (receiver, transmitter, PLL, balun, and antenna switch).
3. **RF** – The radio frequency front-end includes the RF receiver, RF transmitter, clock generator (PLL), balun, and switch. This section handles the actual analog signal transmission and reception for both Wi-Fi and Bluetooth.
4. **RTC / PMU** – Real-time clock memory and the power management unit that orchestrates all sleep states for power efficiency.
5. **Cryptographic Accelerators** – Hardware **SHA**, **AES**, **RSA**, **HMAC**, **RNG**, digital signature, and **XTS-AES-128** flash encryption engines for secure networking and data protection.
6. **Peripherals and Sensors** – Includes **GPIO**, **SPI**, **I²C**, **I²S**, **UART**, **LED PWM**, **GDMA**, **TWAI**, **RMT**, **USB Serial/JTAG**, **ADC**, **timers**, **temperature sensor**, etc.

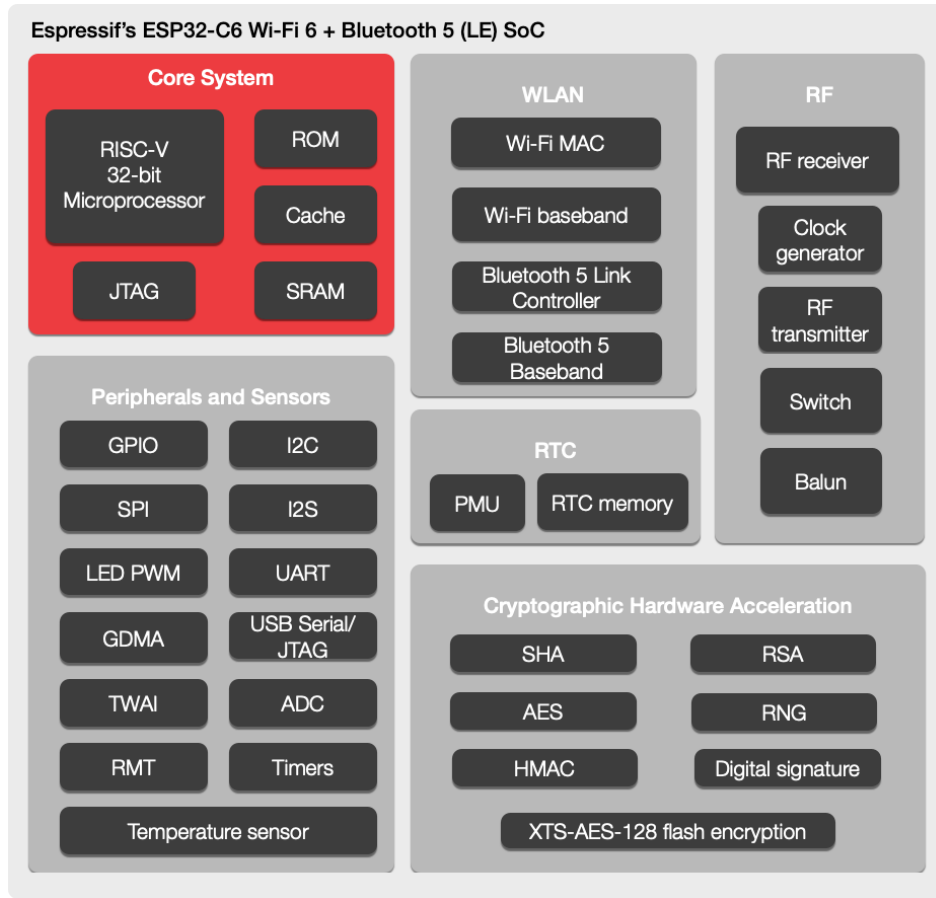


Figure 3.1.: Functional block diagram of the ESP32-C6 SoC (adapted from Espressif's product announcement[2]).

In the *active* state every domain above is powered and clocked: the CPU executes instructions at 160 MHz, the radio can transmit or receive, peripherals are fully responsive and cryptographic blocks accelerate TLS hand-shakes. While this mode delivers maximum performance and a very high throughput thanks to the Wi-Fi 6 PHY, it also consumes very high current making it unsuitable for long-term battery operation in a WBAN. The rest of this chapter therefore analyses how modem-sleep and light-sleep selectively switch off the RF front-end or the entire core system to cut average current without breaking the TCP link.

3.1.1. ESP-only network

The current ESP32-C6 SDK cannot create a Wi-Fi 6 **soft-AP**; an all-ESP topology must utilize Wi-Fi 5 (802.11n). Even when using this older Wi-Fi mode, the MCU still provides

3. Theory / Algorithms

several power-saving options. For the present study we focus on *two*: **modem-sleep**, which turns off only the RF front-end, and **light-sleep**, which halts the entire SoC except the RTC.

The choice of these two modes is guided by a combination of network conditions and application requirements.

1. **Minimise average current** so the node can operate for extended periods from a small battery.
2. **Preserve the TCP connection.** The data stream relies on a persistent TCP connection to ensure stable, lossless communication. Therefore, the board must keep the socket continuously open, avoiding reconnections and potential data loss

we will now focus on the theory behind this two techniques

Modem-Sleep

The first power-saving technique selected for this network configuration is **modem-sleep**.

Modem-sleep is a dynamic power optimization mode in which the ESP32-C6 automatically powers down the radio frequency (RF) components of the chip, such as the transmitter and receiver when they are not actively needed. During this time, the main digital subsystems, including the CPU, SRAM, and most peripherals, remain fully operational.

Unlike light-sleep, modem-sleep allows the device to maintain an active Wi-Fi association with the access point (AP). This is achieved by keeping the MAC layer and internal timers running, enabling the station to wake the RF circuitry just in time to receive beacon frames and buffered packets. In other words, the device does not fully disconnect or deauthenticate from the network, which is essential for protocols like TCP that rely on persistent socket connections.

The ESP32-C6 SDK provides two sub-modes of modem-sleep:

- **Minimum modem-sleep** – the radio sleeps only when no data is being transmitted or received, and wakes frequently to check for traffic and stay in sync with the AP.
- **Maximum modem-sleep** – the radio sleeps more aggressively and skips some beacons, which further reduces power consumption but increases the risk of losing the connection.

3. Theory / Algorithms

In this thesis, only **minimum modem-sleep** is used. This choice ensures that the device stays reliably connected to the AP, preserving the TCP socket and preventing packet loss. Using maximum modem-sleep would be counterproductive in this context, as missing too many beacons could lead to disassociation and require a full reconnection.

Modem-sleep offers a practical compromise between performance and energy savings. It is particularly well-suited for WBAN applications, where continuous connectivity must be preserved but full-speed operation is not required at all times.

In particular, the **minimum modem-sleep** mode can reduce current consumption significantly during idle periods, typically lowering it to a range between **2 and 50 mA**, depending on system activity and radio traffic. This dynamic behaviour allows the device to wake up just in time to handle incoming data, while avoiding unnecessary power drain when no transmission is taking place. As a result, modem-sleep enables substantial energy savings without sacrificing the always-on nature of the network connection.

Light-Sleep

Light-sleep mode on the ESP32-C6 is an advanced power-saving state in which the main system clock, high-frequency domains (including the CPU, most peripherals, and the Wi-Fi MAC layer), and digital peripherals are all disabled. Only the RTC (Real-Time Clock) domain remains powered. This configuration enables the ESP32-C6 to achieve a drastic reduction in power consumption, typically to around 0.8 mA, while keeping RAM and peripherals in a paused state, so the system can quickly continue after waking up. During light-sleep, code execution is paused and the device cannot immediately send or receive data. Wake-up from light-sleep can be triggered by several sources, including:

- Timer interrupts (for periodic wake-ups),
- External events on GPIO or UART,
- Touch sensors (if supported).

A significant advantage of light-sleep over modem-sleep is its much lower energy usage. However, this comes at the cost of increased latency and potential connection loss. Since the Wi-Fi MAC and radio are powered down in light-sleep, the device cannot receive beacon frames or respond to network traffic while sleeping. If wake-up intervals are not carefully synchronized with the network's DTIM (Delivery Traffic Indication Message) schedule, the device may miss communication windows and risk losing its Wi-Fi connection.

Manual vs. Automatic Light-Sleep:

There are two primary approaches to using light-sleep on the ESP32-C6: *manual* and *automatic*.

3. Theory / Algorithms

Manual Light-Sleep: In manual mode, the application explicitly invokes functions such as `esp_light_sleep_start()` to enter light-sleep, typically after configuring the desired wake-up sources. In this mode, the Wi-Fi and Bluetooth drivers must be disabled before entering sleep, and the device will lose all active network connections, including any ongoing TCP sessions. This approach provides maximum control over sleep timing but is not suitable for applications requiring persistent network connectivity.

Automatic Light-Sleep: In automatic mode, the ESP32-C6 operating system manages entry and exit from light-sleep transparently, based on system idleness and Wi-Fi traffic patterns. When automatic light-sleep is enabled, the Wi-Fi driver coordinates sleep cycles with the access point, allowing the device to wake up periodically to receive beacon frames and maintain its association with the network. In this way, automatic light-sleep can preserve the Wi-Fi connection and, by extension, keep TCP connections alive, provided that the sleep intervals and network parameters are properly configured.

In summary, light-sleep is highly effective for applications with predictable, periodic communication patterns, such as Wireless Body Area Networks (WBANs), where short interruptions in connectivity are acceptable. It provides greater power savings than modem-sleep and is ideal for low-duty-cycle scenarios. However, for applications requiring continuous, uninterrupted TCP connectivity, only automatic light-sleep should be considered, for this reason the implementation that will be discussed later uses automatic light sleep. Just as Modem Sleep includes a maximum power-saving variant, Light Sleep also has its own maximum modality, known as Deep Sleep. However, this mode violates one of our design constraints: the board always disconnects from the access point (AP), which may lead to potential data loss.

A summary of the available sleep modes and their characteristics is presented in Table 3.1.

Modality	RF	CPU & Memory	Digital Peripherals	Current [mA]
Active Mode	On	On	On	Up 350
Modem Sleep	Off	On	On	38
Light Sleep	Off	Paused	Paused	0.180
Deep Sleep	Off	Off	Off	0.007

Table 3.1.: Power Modes and Characteristics

3.1.2. Non ESP-only Network

Why an External 802.11ax Access Point is Required

Currently, the ESP-IDF SDK for the ESP32-C6 supports Wi-Fi 6 exclusively in *station mode*. This limitation means that an ESP32-C6 cannot yet operate as a Wi-Fi 6 Access Point (AP), making it impossible to create a fully ESP32-C6-based network that utilizes

Wi-Fi 6 features. Consequently, an all-ESP32-C6 network will use Wi-Fi 5 (802.11n), a Wi-Fi type lacking critical functionalities such as Target Wake Time (TWT), a key feature for power optimization. Therefore, all experimental setups involving TWT in this thesis rely on an external commercial router capable of providing full 802.11ax AP functionality.

Target Wake Time (TWT)

Basic Concept and Individual Target Wake Time (TWT)

Target Wake Time (TWT) is a core feature introduced in Wi-Fi 6 (IEEE 802.11ax) to significantly improve power efficiency. With TWT, a station (STA) and its access point (AP) negotiate precise *service periods*: specific time slots during which the STA wakes to transmit or receive data. Outside these scheduled intervals, the STA can remain in a low-power sleep state, drastically reducing average energy consumption.

Wi-Fi 6 supports two main TWT modes:

- **Broadcast TWT (B-TWT):** The AP defines and advertises a common wake schedule that all participating stations follow. Each STA synchronizes to the parameters set by the AP, simplifying management but limiting individual customization.
- **Individual TWT (I-TWT):** Each STA independently negotiates its own wake schedule with the AP, specifying the interval, offset, and minimum awake duration that best fit its application needs. This allows for highly tailored power management per device.

This project adopts the Individual TWT (I-TWT) approach. By enabling per-device negotiation, each ESP32-C6 node can optimize its own duty cycle: for example, a low-traffic environmental sensor might sleep for tens of seconds between wake-ups, while a latency-sensitive actuator can request much shorter intervals. This flexibility allows the system to save power based on the needs of the application, without reducing overall network performance.

Configuration Parameters for Target Wake Time (TWT)

The configuration of Target Wake Time (TWT) relies on several key parameters negotiated between the station (STA) and the access point (AP). These parameters define when and how often a device wakes up to communicate, directly impacting both power efficiency and network performance.

Figure 3.2 illustrates the structure of the TWT Parameter Set and the TWT Request Type fields. The main fields are:

3. Theory / Algorithms

- **Request Type:** Specifies the type of TWT operation and negotiation details.
- **Target Wake Time:** Indicates the precise time when the STA should wake up for communication.
- **TWT Group Assignment:** Optionally assigns the STA to a group with a shared schedule.
- **Nominal Minimum TWT Wake Duration:** Sets the minimum duration the STA remains awake during each service period.
- **TWT Wake Interval Mantissa and Exponent:** These two fields together determine the interval between wake-ups. The mantissa is a 16-bit (2-byte) field, while the exponent is a 5-bit field. The actual interval is calculated as:

$$\text{TWT Wake Interval} = \text{Mantissa} \times 2^{\text{Exponent}}$$

- **TWT Channel:** Specifies the channel used for the service period.
- **NDP Paging (optional):** Used for paging operations, if needed.

The *TWT Request Type* field 3.2 is subdivided into several bits, including the TWT request flag, setup command, flow type, flow identifier, and the wake interval exponent. This structure allows for fine-grained control over the negotiation and behavior of TWT sessions.

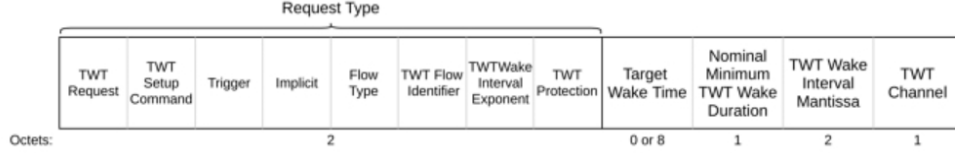


Figure 3.2.: TWT Parameter Set and Request Type field formats. The TWT Wake Interval Mantissa is a 2-byte (16-bit) field, and the TWT Wake Interval Exponent is a 5-bit field. Together, they define the interval between scheduled wake-ups.

Careful configuration of these parameters allows each device to optimize its wake/sleep schedule based on its application needs.

Sleep Modalities Utilized with TWT

TWT itself does not constitute a sleep mode; rather, it enables the station and access point to negotiate predictable sleep and wake intervals, thereby facilitating more consistent low-power operation. Within each negotiated sleep period, the device must select an appropriate internal sleep mode to maximize energy savings.

3. Theory / Algorithms

As discussed earlier, only **minimum modem-sleep** and **light-sleep** are suitable for this use case. Of those, **light-sleep** is preferred during TWT-negotiated sleep intervals, as it achieves significantly lower current consumption (typically around 0.8 mA or even lower when applied with TWT) compared to modem-sleep. Additionally, light-sleep allows the CPU clock frequency to be reduced during idle periods, further minimizing power draw. Upon the scheduled TWT wake-up, the device can promptly resume operation, maintaining both energy efficiency and network responsiveness.

Limitations Introduced by External Routers

Since TWT is a relatively new technology, many routers and other network devices either lack support for it entirely or only support a limited subset of its configuration options. As a result, employing a commercial external 802.11ax router introduces several practical limitations in the TWT configuration:

- **Maximum Wake Interval Restrictions:** Consumer-grade routers often impose specific limitations on the permissible Target Wake Time parameters, particularly for very short or very frequent wake intervals. Some routers may reject these settings outright, causing the TWT negotiation to fail and potentially destabilizing the network connection.
- **Vendor-Specific Parameter Handling:** Another observed limitation relates to the configuration of the wake interval unit. The duration of the wake period is determined by multiplying the wake interval by a unit value, where a unit of 0 represents 256 μ s and a unit of 1 represents 1024 μ s. Certain routers do not permit the selection of unit 1, thereby significantly restricting the maximum possible wake duration and limiting the flexibility of TWT scheduling.

Despite these constraints, integrating TWT with the ESP32-C6's light-sleep mode still results in substantial reductions in average current consumption—often achieving sub-milliamperere levels—while reliably maintaining TCP connections. This makes the ESP32-C6, in combination with TWT, particularly attractive for battery-powered WBAN applications that require predictable, periodic communication with minimal energy overhead.

Table 3.2 summarize the different technology available for different setup

Table 3.2.: Power-saving levers evaluated on the ESP32-C6 under two network set-ups

Network under test	Wi-Fi type	Modem sleep	Light sleep	TWT
All-ESP (soft-AP + client)	Wi-Fi 5 / 802.11n	Yes	Yes	No
ESP client + Wi-Fi 6 router	Wi-Fi 6 / 802.11ax	Yes	Yes	Yes

3.2. Time Synchronization

While much attention has been devoted to power optimization in Wireless Body Area Networks (WBANs), time synchronization is an equally fundamental aspect that must not be overlooked. Accurate time synchronization is essential for ensuring that data collected from multiple sensor nodes can be reliably combined, interpreted, and utilized. Without proper synchronization, sensor readings may become misaligned in time, leading to inconsistent, misleading, or even catastrophic results during data analysis or aggregation.

In a typical WBAN, each node operates based on its own local clock, which is susceptible to drift over time due to factors such as temperature fluctuations, supply voltage variations, and component aging. Even minor discrepancies in clock rates can accumulate and result in significant temporal offsets between nodes. Such offsets can disrupt the temporal ordering of events, hinder the correlation of measurements, and ultimately compromise the integrity of applications like medical monitoring, activity recognition, and event detection.

To address these challenges, this chapter explores one of the most robust and widely adopted solutions: the built-in Timing Synchronization Function (TSF) specified in the IEEE 802.11 standard. Leveraging TSF provides a practical and effective means of achieving precise time synchronization across all nodes in a WBAN.

3.2.1. IEEE 802.11's Built-in Timing Synchronization Function

Time synchronization is a fundamental requirement in wireless networks, particularly in Wireless Body Area Networks (WBANs), where data collected from multiple sensor nodes must be temporally aligned for accurate analysis and interpretation. To address this, our system leverages the built-in *Timing Synchronization Function (TSF)* provided by the IEEE 802.11 standard, a widely adopted protocol in WLAN environments.

3. Theory / Algorithms

Suitability for Star Topology The TSF mechanism is especially well-suited for networks with a star topology, where a central Access Point (AP) manages communication with multiple client nodes. In this configuration, all nodes connect directly to a single AP, which serves as the network's central time reference.

How TSF Works

- Each device (station) in the network maintains an internal TSF timer, which counts in microsecond increments.
- The AP periodically broadcasts beacon frames containing a timestamp that reflects its current TSF timer value.
- When a client node receives a beacon, it compares the timestamp in the beacon with its own TSF timer.
- If the beacon's timestamp is ahead of the node's local timer, the node updates its timer to match the AP, ensuring synchronization across all devices.

This process is repeated at regular intervals, allowing client nodes to continuously correct for any clock drift or imperfections that may arise over time. As result, all nodes remain closely synchronized with the AP, which is essential for coordinated communication, power management, and time-sensitive applications such as those found in WBANs.

Firmware Architecture

4.1. System Architecture

4.1.1. System Characterization

As previously discussed, the proposed WBAN adopts a star topology, where a central master node acts as the Access Point (AP) and multiple client nodes communicate exclusively with the AP. While the overall network structure remains consistent, leveraging the features of Wi-Fi 6 required distinguishing between two central node configurations.

Specifically, two main system configurations were explored:

- **ESP-to-ESP:** In this configuration, both the AP and the client nodes are ESP devices (e.g., ESP32-C6), communicating over Wi-Fi 5. This setup allows for complete control over both ends of the communication link, making it ideal for low-level protocol experimentation and direct firmware optimization.
- **ESP-to-Router:** In this scenario, the ESP nodes operate as clients connected to a commercial Wi-Fi 6 router or access point that supports Target Wake Time (TWT). This arrangement enables the evaluation of advanced power-saving features introduced with Wi-Fi 6, such as TWT negotiation and scheduled communication windows.

4.1.2. Firmware Structure

The analysis begins with the scenario in which the network is composed exclusively of ESP devices. In this configuration, both the Access Point (AP) and the client nodes require dedicated firmware. The foundational code for both roles was originally developed by a

4. Firmware Architecture

previous student as part of their thesis work [18], providing a solid base for subsequent enhancements.

This chapter is limited to outlining the basic aspects of the implementation. The duty cycle approach and the corresponding results will be presented and discussed in the following chapter.

4.1.3. All-ESP Network

This section describes the main firmware components used to implement the theoretical concepts introduced earlier.

Access Point Firmware Overview

The Access Point (AP) functions as the central coordinator in the star topology network. Its core responsibilities include managing communication with all client nodes, broadcasting synchronization beacons, and maintaining network synchronization. The AP firmware, initially developed by a previous student, was designed to ensure reliable connectivity and coordination across the network.

Figure 4.1 illustrates a high-level overview of the AP firmware architecture and its principal functional blocks.

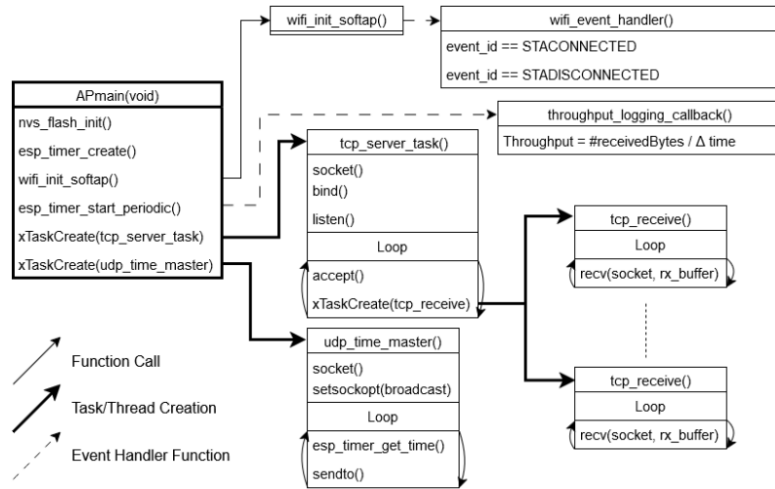


Figure 4.1.: High-level architecture of the Access Point firmware (developed by a previous student).

No significant modifications were made to the AP firmware in this work, apart from the removal of the `udp_time_master` component.

4. Firmware Architecture

Client Firmware Overview

Client nodes are responsible for collecting sensor data and maintaining communication with the AP. The main contribution in this work concerns the development and optimization of the client firmware, with particular focus on the *tcp_task*, which manages TCP communication with the AP. The initial version of this code was also provided by the previous student.

Figure 4.2 presents a high-level diagram of the client firmware architecture and its main functional modules.

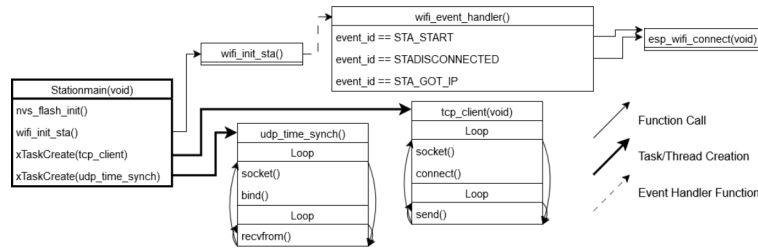


Figure 4.2.: High-level architecture of the client firmware.

The client firmware is the area where the most significant modifications were introduced. In this version, the *udp_client* component was removed, and the majority of changes focused on the *tcp_client* task.

- Modem Sleep:** To enable modem sleep, the appropriate power management library must be included, after which the modem sleep mode can be activated using a specific command. This command accepts parameters that determine the desired power-saving level: no power save, modem minimum, or modem maximum. In this implementation, the *modem minimum* mode was selected to balance energy savings with network responsiveness.
- Light Sleep:** The implementation of light sleep required a more sophisticated approach. The initial version of the firmware relied on manual sleep management, which proved to be complex and difficult to maintain. The transition to an automatic method considerably simplified this process. In this approach, light sleep is first configured in the main code, including parameters such as the clock frequency (which, for the ESP32-C6, ranges from a minimum of 40 MHz to a maximum of 160 MHz). Once configured, the board automatically enter light sleep during idle periods, lowering the cpu frequency and putting in a "sleep state" peripheral component, thereby improving energy efficiency without manual intervention.

4.1.4. ESP-to-Router

In the ESP-to-router configuration, the ESP devices act as client nodes connecting to a commercial Wi-Fi 6 router. Unlike the ESP-to-ESP scenario, the router operates as a “black box” and cannot be directly programmed or modified. The only configuration required on the router side was to connect it to a computer and use the `ncat` utility to open a TCP socket, enabling communication with the ESP clients.

On the client side, the implementation presented additional challenges. The starting point for the firmware was the example code provided in the official Espressif repository¹. This reference implementation demonstrated how to establish a connection to a Wi-Fi 6 router and negotiate a Target Wake Time (TWT) session. However, the example was limited to basic connectivity and TWT negotiation, without support for application-level TCP communication or advanced power management.

To address these limitations, two major enhancements were introduced:

- **TCP Communication Task:** A dedicated `tcp_task` was developed to manage reliable data exchange between the ESP client and the server (running on the computer connected to the router). This task handles socket creation, connection management, data transmission, and error handling, enabling robust and continuous communication over the established TCP link.
- **Light Sleep Configuration:** In order to fully leverage the power-saving capabilities of Wi-Fi 6 and TWT, the client firmware was extended to support automatic light sleep. This required configuring the relevant parameters (such as clock frequency and wakeup sources) and integrating the sleep logic with the TWT negotiation process. As a result, the ESP client can automatically enter light sleep during idle periods, waking up in synchronization with the negotiated TWT schedule to maintain efficient communication with the router.

These modifications enabled the ESP clients to operate efficiently in a commercial Wi-Fi 6 environment, taking advantage of both advanced scheduling (TWT) and low-power operation, while maintaining reliable TCP communication with the server.

¹<https://github.com/espressif/esp-idf/tree/master/examples/wifi/itwt>

Design Implementation

5.1. Power Optimization

The core of the power optimization strategy is based on the concept of a *duty cycle*. In the context of embedded systems and wireless communication, the duty cycle refers to the proportion of time a device spends in its active (awake or transmitting) state relative to the total duration of an operational cycle. This can be mathematically expressed as:

$$\text{Duty Cycle } (D) = \frac{T_{\text{ON}}}{T_{\text{period}}} \times 100\% \quad (5.1)$$

where T_{ON} is the time the device remains active, and T_{period} is the total length of the cycle (active time plus sleep time).

By adjusting the ratio of active to sleep time, it is possible to balance energy consumption with the application's requirements for latency and responsiveness. For instance, a duty cycle of 20% means the device is active for 20% of the cycle and in a low-power or sleep state for the remaining 80%.

In this work, several latency are explored to address different application scenarios:

- **100 ms:** suited for low-latency applications,
- **1 second:** appropriate for biosignal monitoring,
- **100 seconds:** targeting slow-changing signal applications.

By systematically varying the duty cycle and its period, the trade-off between power savings and system performance can be quantitatively evaluated for each use case.

For both the all-esp and not network the flow diagram is the following 5.1:

5. Design Implementation

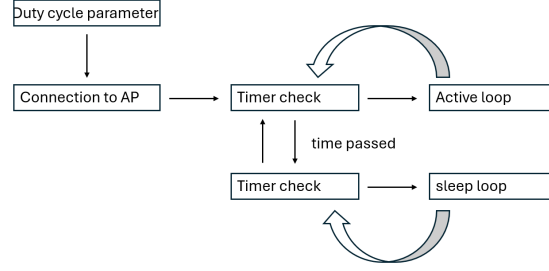


Figure 5.1.: Time based cycle flow diagram

5.1.1. Current Profile Analysis in Light Sleep Mode

The current profile shown in Figure 5.2 illustrates the behavior of the system in light sleep mode for an all-ESP network, configured with a period $T = 100$ ms and a duty cycle of 40%.

The measured current statistics are as follows:

- **Minimum:** 15.56 mA
- **Average:** 105.55 mA
- **Maximum:** 368.03 mA
- **Peak-to-Peak:** 352.47 mA

As observed in the figure, the current waveform clearly demonstrates a 40 ms active data transmission period, followed by 60 ms of low-power operation. This pattern matches the intended 40% duty cycle, confirming that the code is functioning as expected. The significant difference between the minimum and maximum current values reflects the transitions between active transmission and light sleep states, characteristic of energy-efficient ESP network operation.

5. Design Implementation

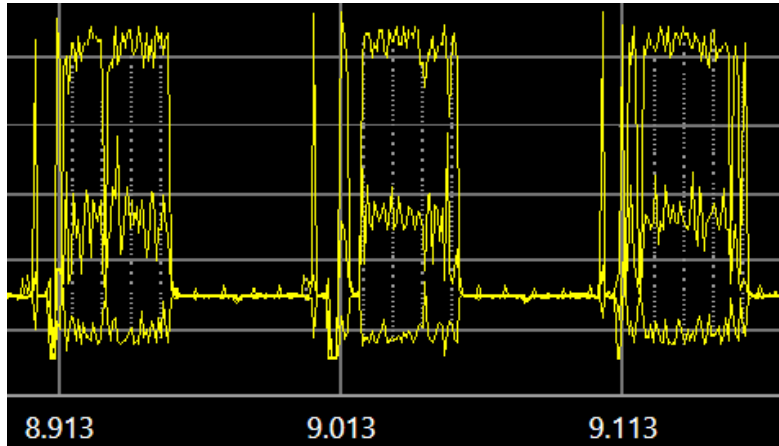


Figure 5.2.: Example of current consumption.

5.1.2. All-ESP

Modem Sleep

The implementation of modem sleep is based on precise timing to alternate between active and sleep phases within each duty cycle. This is achieved by saving a reference timestamp at the start of each cycle using the `esp_timer_get_time()` function, which provides the current system time in microseconds.

At the beginning of each duty cycle, the current time is stored as the cycle reference. The device then enters the **active phase**, during which it remains fully awake and continuously transmit packet of 1460 byte. The duration of this phase is controlled by continuously checking the difference between the current time and the reference timestamp. As long as this difference is less than the predefined active interval, the device remains in the active state. During this period, the call

```
esp_wifi_set_ps(WIFI_PS_NONE)
```

is used to ensure that the Wi-Fi radio is fully powered and responsive.

Once the elapsed time in the active phase exceeds the configured value, the device transitions to the **sleep phase**. The end of the duty cycle period is computed by adding the total duty cycle duration to the original reference timestamp. During the sleep phase, the device enters modem sleep mode by invoking

```
esp_wifi_set_ps(WIFI_PS_MIN_MODEM)
```

5. Design Implementation

which significantly reduces energy consumption by disabling the Wi-Fi radio when it is not needed.

The device remains in the sleep phase until the total duty cycle period has elapsed, as determined by checking when the difference between the current time and the initial reference timestamp exceeds the full duty cycle duration. At that point, a new cycle begins by updating the reference time and repeating the process.

By doing so we can analyze the performance of the power save technique with for different test case

Light Sleep

The implementation of light sleep is based on both an initial configuration step and a precise timing mechanism within the main operational loop. Additionally to light sleep also the modem sleep is used in order to have even more power efficiency we used a combination of light-sleep + modem min

Main Function Before entering the main loop, light sleep is enabled by configuring the ESP32's power management system. This is done by defining a power management configuration structure, where the minimum and maximum CPU frequencies are set (for example, 40 MHz and 160 MHz), and the light sleep feature is activated. With this configuration, the ESP32 is allowed to automatically enter light sleep whenever the system is idle and no tasks require immediate CPU usage.

TCP Task In the `tcp_task`, the device alternates between an active phase and an idle phase, both managed using the high-resolution timer (`esp_timer_get_time()`). At the start of each duty cycle, the current time is recorded as a reference. The implementation of the modem sleep is done as explained in the last subunit.

During the **active phase**, the Wi-Fi radio is set to full power save off by calling

```
esp_wifi_set_ps(WIFI_PS_NONE)
```

This ensures the radio remains fully operational for reliable data transmission. The device remains in this state until the elapsed time, measured as the difference between the current time and the reference timestamp, reaches the predefined active interval.

At the end of the active phase, the device transitions to the **idle phase**. Here, the Wi-Fi radio is set to modem sleep mode by calling

```
esp_wifi_set_ps(WIFI_PS_MIN_MODEM)
```

5. Design Implementation

The real implementation of the light sleep is done in the wait loop in which we implement the idle period, the application invokes a delay function (such as `vTaskDelay`) for the required duration.

With light sleep enabled in the initial configuration, the ESP32 will automatically enter light sleep during these idle periods, provided no other tasks are ready to run. The use of the delay function is essential, as it informs the operating system that the CPU can be suspended, thereby triggering the automatic entry into light sleep mode. The system will remain in light sleep until the delay expires or another task becomes ready, at which point normal operation resumes.

This approach ensures precise control over the timing of active and idle phases, while leveraging the ESP32's automatic light sleep capability to minimize energy consumption during idle periods, all without requiring manual intervention to enter or exit light sleep. The combination of explicit power state transitions for the Wi-Fi radio and automatic CPU light sleep allows for systematic evaluation of power savings under different duty cycle configurations.

5.1.3. Non All-ESP Network

After completing the modem sleep and light sleep implementations, the next step was to incorporate Target Wake Time (TWT) functionality. In this context, TWT serves as an advanced alternative to the basic “loop with timer” approach for controlling device activity and power consumption.

Target Wake Time (TWT)

The implementation of TWT is based on the official example provided in the Espressif GitHub repository. TWT is a power-saving mechanism introduced in Wi-Fi 6 (802.11ax) networks, which allows client devices to negotiate specific wake-up times with the access point, thereby reducing unnecessary energy consumption.

TWT Negotiation and Configuration The TWT negotiation process begins after the device connects to a Wi-Fi 6 router and obtains an IP address. The configuration is defined through a dedicated structure, which specifies several critical parameters:

- **Flow Type:** Determines whether the schedule is announced or unannounced. Typically, it is set to announced, so the device wakes up at predetermined times known to the access point.
- **Trigger:** Enables or disables the use of trigger frames from the access point to initiate data transmission during wake periods.

5. Design Implementation

- **Wake Duration:** Specifies the duration the device remains awake during each service period. This value is scaled according to a unit parameter, allowing for flexible configuration.
- **Wake Interval:** Defined by a mantissa and exponent, the actual interval is calculated as $\text{mantissa} \times 2^{\text{exponent}} \times 1024$ microseconds.

Unlike the previous manual timer-based approach, these parameters allow the duty cycle to be implemented directly through the TWT negotiation. However, there is a limitation in the maximum configurable wake duration. While the sleep interval (regulated by the mantissa, exponent, and unit) can be set to extremely long periods (up to several years), the wake duration is constrained by the protocol to only 8 bits. Depending on the unit selected (either 256 or 1024 microseconds), the maximum wake duration is $255 \times 1024 = 261,120 \mu\text{s}$, or approximately 256 ms.

Due to this constraint, in this implementation, only the case with a period of 100 ms was used, and an additional configuration was tailored for a wake duration of 250 ms. These settings ensure that the device’s active period remains within the protocol’s allowable range.

Once the awake and sleep intervals were chosen and light sleep was configured, the next step was to enable the TCP connection.

TCP Task Implementation The main challenge in this phase was ensuring that the TCP task transmits data strictly within the designated awake periods, avoiding transmission during sleep intervals. Failing to do so could result in the loss of synchronization with the TWT schedule, leading to missed transmission opportunities and a significant drop in throughput.

To address this, the implementation leverages the trigger field in the TWT configuration. Each time the device wakes up for a scheduled service period, a flag is set, indicating that the TCP task can begin data transmission. Additionally, a timer is used to ensure that data transmission occurs only within the allocated wake window, preventing any attempt to send data outside the permitted interval. This approach maintains strict alignment with the TWT schedule and ensures reliable and efficient data transfer during the active periods.

Power Optimization

In traditional Wi-Fi networks (802.11 b/g/n), even with light sleep enabled, the ESP32 must wake up at every beacon interval—typically every 100 ms—to receive Delivery Traffic Indication Map (DTIM) beacons from the access point. Each beacon forces the chip’s RF and clock domains to power up for several milliseconds, causing periodic current

5. Design Implementation

spikes. Although the system returns to light sleep between beacons, these frequent wake-ups constrain the average idle current to approximately 0.8–1 mA. Furthermore, because the device must remain ready to wake up for each DTIM beacon, it cannot reduce its clock frequency to the minimum 4 MHz, preventing it from reaching the lowest possible power consumption in this mode.

With the introduction of Target Wake Time (TWT) in Wi-Fi 6 (802.11ax), the device and access point can negotiate precise wake-up schedules. This allows the ESP32 to remain in light sleep continuously between scheduled service periods, ignoring all beacons. The access point buffers traffic and communicates with the device only during the agreed TWT window. As a result, the ESP32’s idle current drops to the true light sleep minimum—typically around 50 μ A—since it no longer needs to wake for every beacon. Additionally, this continuous sleep enables the device to lower its clock frequency to the minimum 40 MHz, which is not possible with manual light sleep due to the requirement to wake for DTIM beacons.

This protocol-level optimization significantly extends battery life compared to standard light sleep, while maintaining a persistent Wi-Fi connection and reliable data delivery. TWT transforms Wi-Fi from a duty-cycled receiver into a truly scheduled, low-power radio, unlocking the full potential of light sleep for IoT and battery-powered applications.

Router Limitation

A significant limitation when deploying TWT in real-world scenarios is the lack of widespread and consistent support among commercial Wi-Fi 6 routers. As TWT is a relatively recent addition to the 802.11ax standard, many routers either do not support it at all or implement it only partially, often with proprietary restrictions or incomplete firmware.

In this thesis, two different routers were tested to evaluate TWT functionality. The first, an Asus RT-AX57, exhibited very limited TWT support: it rejected most of the negotiated configurations, making it unsuitable for flexible experimentation. The second, a TP-Link Archer AX58, demonstrated much better compatibility, reliably supporting short and frequent wake windows. However, it also presented a major limitation: it did not fully support the “wake interval unit” parameter. Specifically, the router only accepted the unit value set to zero, rather than one, which effectively limited the minimum wake interval to 255×256 microseconds (about 65 ms). This restriction prevented the use of longer or more flexible TWT intervals that would have been possible with full standard compliance.

As a result, the experimental evaluation in this work was constrained by the capabilities of the available routers, highlighting the need for more mature and standardized TWT implementations in consumer networking equipment.

5.1.4. Time Synchronization Function

After the implementation and evaluation of various power optimization techniques, the next critical objective was to achieve precise time synchronization across multiple acquisition boards. Accurate time alignment is essential for distributed biopotential recording systems, where data from several nodes must be temporally correlated for downstream analysis.

5.1.5. Packet Header

Once a robust time synchronization mechanism is in place, the next challenge is to ensure that the Access Point (AP) can accurately determine the precise timestamp at which each packet was sampled. This capability is crucial for aligning and analyzing data received from multiple nodes in a Wireless Body Area Network (WBAN), where precise temporal correlation is essential.

To address this, a custom packet header was designed to encapsulate all necessary metadata for synchronization and data integrity. The structure of the header is as follows:

- **Board ID (1 byte):** Identifies the source node, enabling the AP to distinguish between data streams from different clients.
- **Sequence Number (4 bytes):** A monotonically increasing counter assigned before transmission. This allows detection of lost or out-of-order packets, even though TCP should generally prevent packet loss.
- **Payload Length (2 bytes, big-endian):** Specifies the size of the payload, allowing the receiver to parse incoming data correctly.
- **TSF Timestamp (8 bytes):** Records the exact time (in microseconds) when the data was sampled, based on the network-synchronized Timing Synchronization Function (TSF) clock. This field is critical for precise temporal alignment across all nodes.

The total header size is 15 bytes [5.3], and it is prepended to every data packet before transmission. This design ensures that each packet contains all the information required for source identification, packet ordering, payload size verification, and high-precision time synchronization. Such a structure is particularly important in WBAN applications, where accurate timing and data integrity are fundamental for meaningful analysis.

5. Design Implementation

board_id	sequence	length	time_stamp
1 byte	4 bytes	2 bytes	8 bytes

Figure 5.3.: Structure of the custom packet header used for time-synchronized data collection.

Client-Side Implementation

The client code was extended to support both continuous data streaming and precise event marking. Key aspects include:

- **Payload Construction:** Each packet contains 96 bytes of data, corresponding to 32 ExG channels sampled at 2 kHz, with 3 bytes per channel. This is considerably smaller than the maximum payload (around 1460 bytes), reducing latency and supporting high-frequency transmission.
- **Transmission Rate Control:** To match the biogap’s 2 kHz sampling rate, a busy-wait loop of 0.5 ms is inserted after each packet transmission. This ensures that packets are sent at the correct frequency, emulating real-time streaming.
- **External Trigger Handling:** For synchronization experiments, an external step signal is provided by a waveform generator. This signal is connected to GPIO 18 on each ESP32C6 board. An interrupt service routine (ISR) is configured to detect rising edges on this pin. When a trigger is detected, the ISR records the current TSF timestamp and sets a flag. The main TCP task checks this flag and, upon detection, switches the outgoing packet payload from all zeros (normal state) to all ones (triggered state), embedding the precise trigger timestamp in the header.
- **Timestamping:** Under normal operation, the timestamp field is set immediately before packet construction. In the case of a trigger, the timestamp captured by the ISR is used instead, ensuring minimal timing jitter.

This architecture allows each board to independently mark the occurrence of a shared external event, enabling accurate measurement of inter-device time offsets.

Experimental Setup and AP-Side Handling

For the synchronization experiment, two ESP32C6 development kits were used (due to the lack of accessible GPIOs on the Wi-Fi shield hardware), but the results are expected to generalize. Both boards receive the same step signal from a waveform generator, ensuring simultaneous triggering.

5. Design Implementation

On the access point (AP) or server side, the TCP receiver was modified to:

- Parse and extract the custom header from each incoming packet.
- Check the payload content to distinguish between normal (all zeros) and triggered (all ones) packets.
- Log the timestamp and board ID from each trigger event for subsequent analysis.

This setup allows for the collection of precise timing data from multiple clients in response to a common external event, providing a robust basis for evaluating synchronization accuracy.

Client and AP Synchronization

After validating synchronization between two client devices, the next step was to extend the experiment to include the access point (AP) itself. This was achieved by connecting the trigger signal not only to the clients but also to the AP. On the AP, an interrupt service routine (ISR) was implemented in the same manner as on the clients: the AP monitors its designated GPIO pin for the trigger edge, captures the timestamp using the same TSF-based method, and flags the event for subsequent packet processing.

This unified approach ensures that all devices—both clients and the AP—record the precise moment of the external trigger using their local clocks. By comparing the timestamps collected from each node, it is possible to directly assess the absolute and relative synchronization accuracy across the entire system, including the AP.

This setup, depicted in Figure 5.4, allows for comprehensive evaluation of time synchronization performance, as all relevant nodes in the network (clients and AP) respond to the same external event under identical conditions. It should be noted that, while the figure illustrates the types of components used in the experiment, the specific models shown may not exactly match those employed in the actual setup; however, the functional roles and connections remain representative of the experimental configuration.

Beacon Frequency

Since the Timing Synchronization Function (TSF) relies on periodic beacon frames to broadcast timestamp information across the network, an interesting question arises: can reducing the beacon broadcast frequency—by default set to 100 ms—lead to lower power consumption without compromising time synchronization performance?

Fortunately, the beacon interval is a configurable parameter in most Wi-Fi setups, and can be set to any value within the range of 100 ms to 60,000 ms. To investigate the impact of beacon frequency on both power consumption and synchronization accuracy, a series of experiments were conducted. The beacon interval was systematically varied, and, as

5. Design Implementation

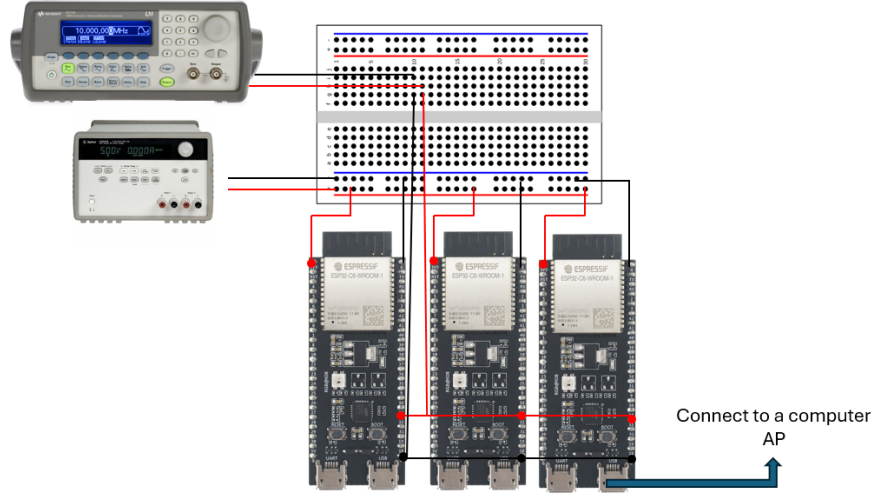


Figure 5.4.: Experimental setup for time synchronization: both ESP32-C6 clients and the AP receive the trigger from the waveform generator. Image assembled in PowerPoint using elements from Keysight E3647A [3], Keysight 33210A [4], ESP32-C6 [5], and breadboard [6].

in previous tests, synchronization performance was monitored. In addition, the access point (AP) was connected to a power analyzer to precisely measure current consumption under each configuration.

This approach allows for a direct evaluation of the trade-off between energy efficiency and synchronization fidelity, providing valuable insights for optimizing WBAN deployments where both parameters are critical.

Chapter 6

Results

This chapter presents the experimental results for the power optimization strategies developed on the ESP32 platform. The evaluation includes both all ESP and non-all ESP network configurations, examining how different power-saving modes and protocol-level features influence energy consumption and system performance. Key metrics such as current consumption, synchronization accuracy using the Timing Synchronization Function (TSF), and throughput are analyzed under various operating conditions. These results provide quantitative insights into the trade-offs and benefits of the implemented optimizations, and offer guidance for future improvements in low-power wireless communication systems.

6.1. Power Optimization

6.1.1. All-ESP

To rigorously evaluate the impact of power optimization strategies, experiments were conducted using three distinct duty cycles across a network composed entirely of ESP-based devices. This approach allowed for a systematic comparison of performance and energy consumption under varying transmission schedules. Each duty cycle was tested independently, and the resulting data provide insight into how different activity patterns affect overall system efficiency and throughput.

The results for each time period are illustrated in the following figures 6.1 6.2 6.3:

6. Results

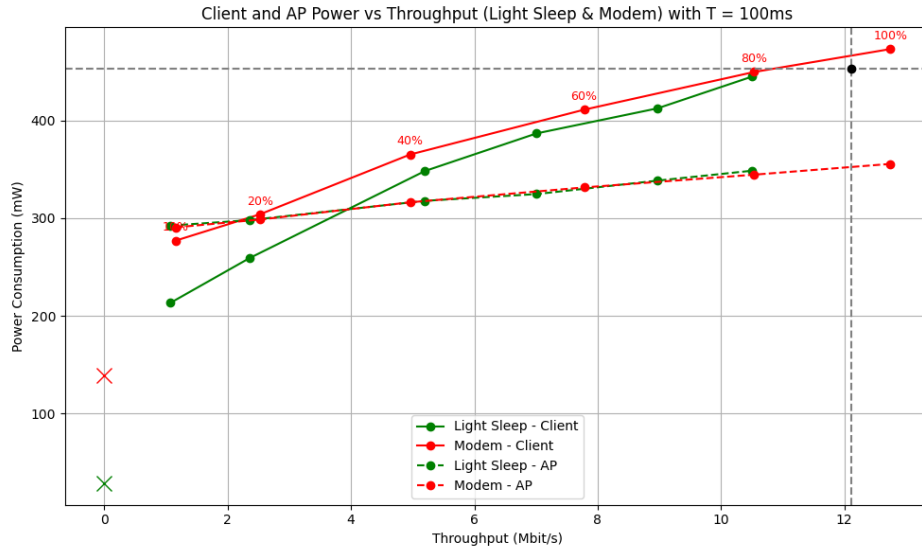


Figure 6.1.: Results for the 100ms total period with different duty cycle, both for AP and client.

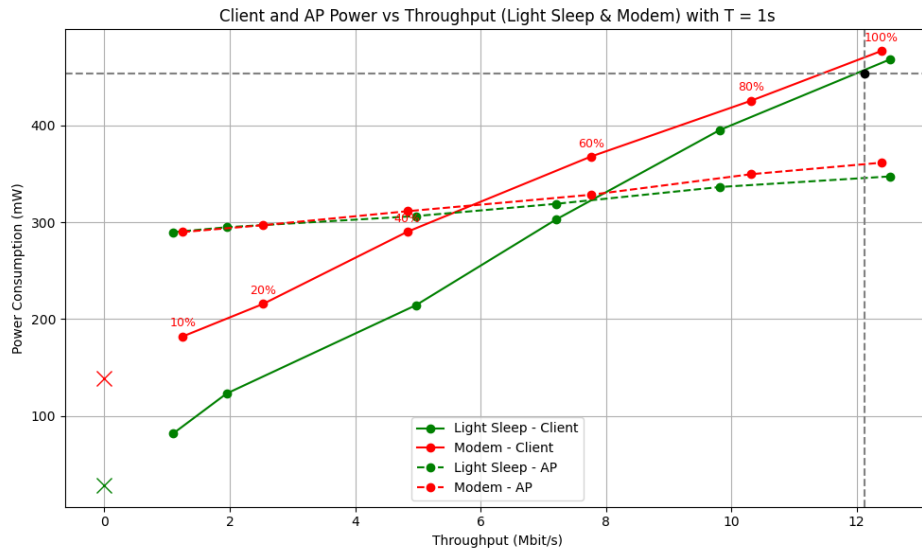


Figure 6.2.: Results for the 1s total period with different duty cycle, both for AP and client.

6. Results

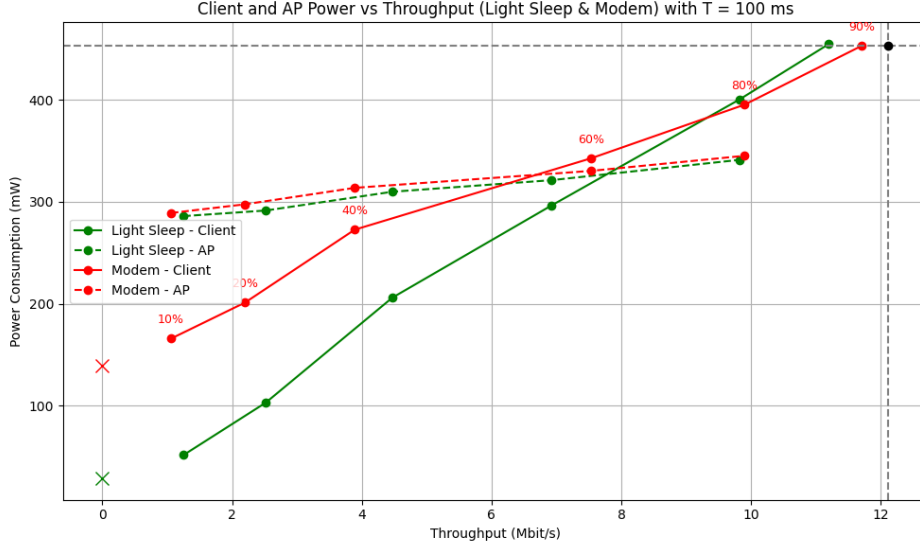


Figure 6.3.: Results for the 100s total period with different duty cycle, both for AP and client.

As anticipated from theoretical considerations, the results show that light sleep consistently outperforms modem sleep, particularly at low duty cycles. This improvement is primarily due to the fact that, with reduced duty cycles, the system can operate at lower clock frequencies, thereby significantly reducing power consumption.

The three graphs presented illustrate the measured current for both the client and the access point (AP) under light sleep and modem sleep modes. Additionally, each plot includes the baseline power consumption at zero duty cycle (idle state) and the maximum throughput achieved by running the code without any power optimization techniques.

It is important to note that, despite efforts to standardize the measurement conditions, some variability in the recorded values was observed. This is largely attributable to the experimental environment: the 2.4 GHz frequency band is heavily utilized and susceptible to external interference, which can introduce bottlenecks and fluctuations in performance. Every attempt was made to minimize these effects and ensure consistency between measurements, but some degree of imprecision remains inherent to the setup.

6.1.2. Non-All-ESP

As with the all-ESP network, experiments were also conducted using a heterogeneous setup where ESP devices communicated through a standard Wi-Fi router. In this configuration, four distinct graphs were generated, each corresponding to a different duty cycle and network condition, including the use of Target Wake Time (TWT) where supported.

6. Results

The results confirm that light sleep continues to outperform modem sleep, especially at lower duty cycles, due to the system's ability to reduce clock frequency and thus average power consumption. Notably, when TWT is enabled, further power savings are observed: TWT allows the device to remain in light sleep for longer, uninterrupted intervals by negotiating precise wake-up times with the router. This leads to a significant reduction in idle current, as the device no longer needs to wake up for every beacon, but only during scheduled service periods.

The four graphs below summarize these findings for the router-based network, including the impact of TWT 6.4 6.5 6.6 6.7:

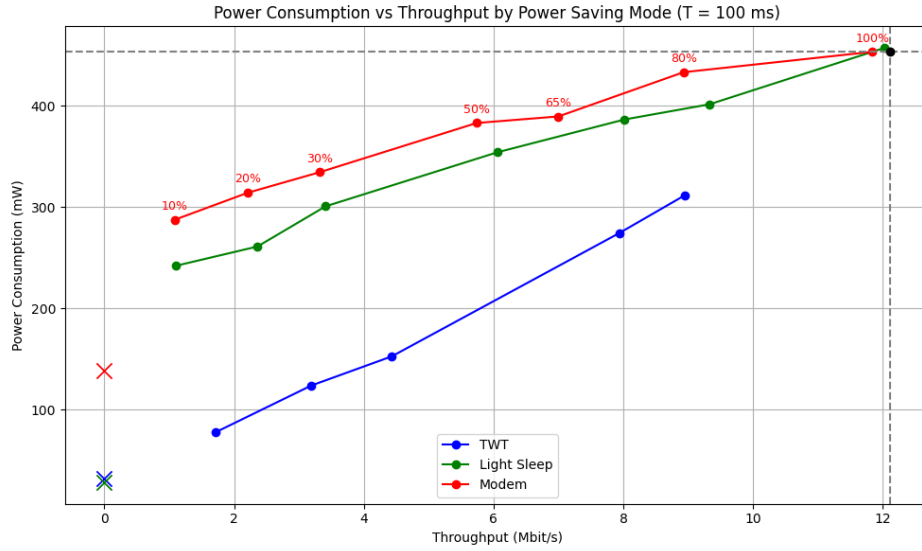


Figure 6.4.: Results for the 100 ms total period with different duty cycles for the client.

6. Results

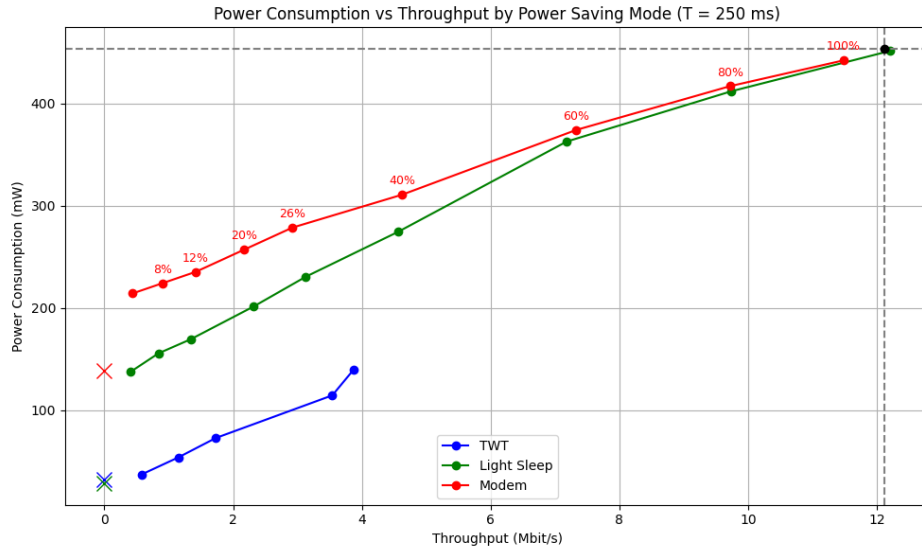


Figure 6.5.: Results for the 250 ms total period with different duty cycles for the client.

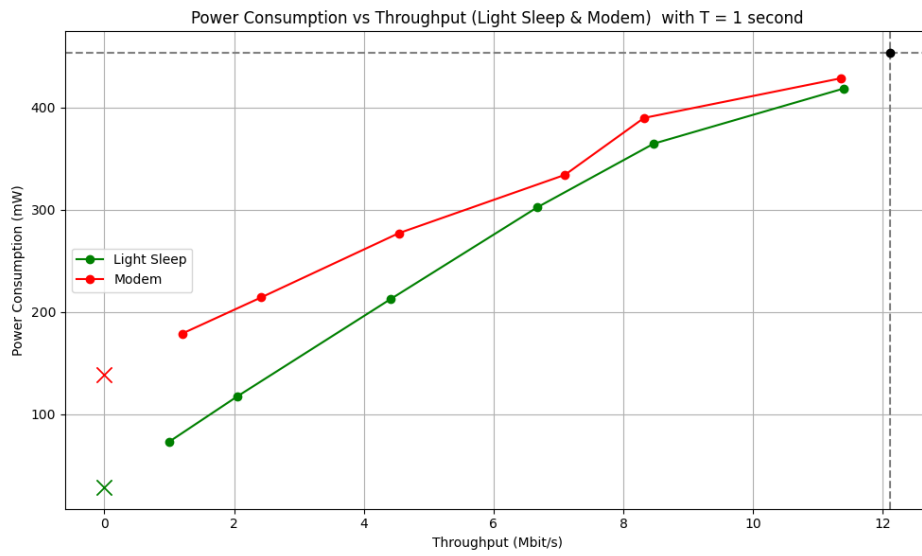


Figure 6.6.: Results for the 1 s total period with different duty cycles for the client.

6. Results

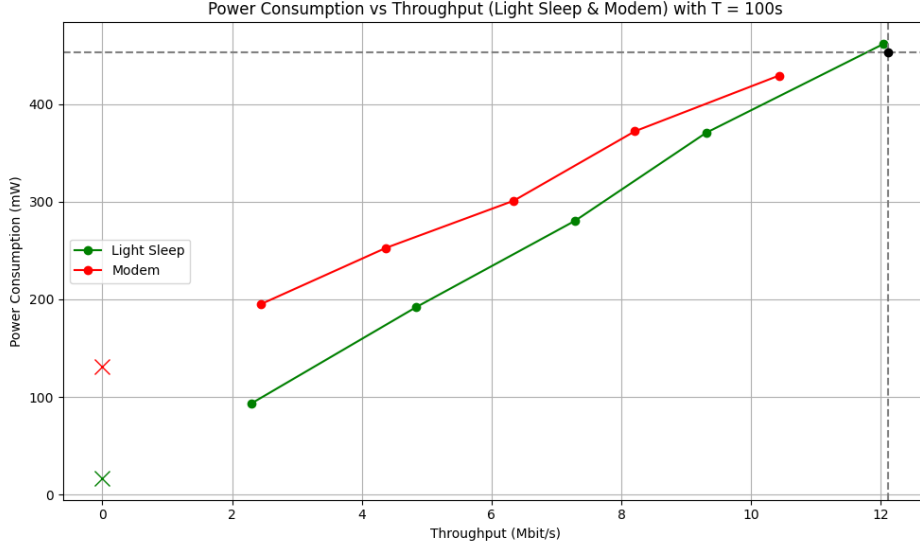


Figure 6.7.: Results for the 100 s total period with different duty cycles for the client.

Each graph reports the measured current for the client in light sleep, modem sleep, and TWT-enabled modes. Additionally, the baseline power consumption at zero duty cycle (idle state) and the maximum throughput achieved without any power optimization are indicated for reference.

It is important to note that, despite careful efforts to maintain consistent measurement conditions, some variability in the results was observed. This is largely due to the experimental environment: the 2.4 GHz Wi-Fi band is heavily used and prone to external interference, which can introduce bottlenecks and fluctuations in network performance. While every attempt was made to minimize these effects and ensure comparability between measurements, some degree of imprecision remains inherent to the setup.

For this case, the time periods considered were 100 ms, 250 ms, 1 s, and 100 s. The first two intervals (100 ms and 250 ms) were mainly used to test TWT performance. As mentioned earlier, router limitations restrict the maximum wake time to approximately 65 ms (256×255 microseconds), which constrained the TWT evaluation to these shorter periods.

The trends for basic modem sleep and light sleep are consistent with those observed in the all-ESP experiments. However, TWT clearly outperforms both modes significantly, as expected due to its optimized wake scheduling.

The longer periods of 1 s and 100 s were used to replicate previous experiments and provide a broader perspective. In these cases, TWT was not considered because the duty cycle would be less than 1%, making its impact negligible.

6.2. TSF

The following results show the evaluation of time synchronization accuracy using the Timing Synchronization Function (TSF). The initial set of tests focused on a set-up with two client devices.

6.2.1. TSF with Two Clients

In this experiment, both clients continuously streamed data to the access point (AP) over a period of approximately 40 minutes. Every 5 seconds, an external trigger was sent to both boards, prompting each to transmit a packet containing a predefined pattern to the AP. This regular triggering enabled precise measurement of synchronization performance between the two clients.

To assess both the precision and the temporal stability of the synchronization:

- Figure 6.8 presents a box plot of the synchronization error between the two clients, providing a statistical overview of the precision, variance, and distribution of the timing differences throughout the experiment. This visualization allows for a clear assessment of the typical synchronization accuracy and the presence of any outliers or variability in the results.
- Figure 6.9 shows the synchronization error as a function of time over the entire 40-minute period. This plot is used to evaluate whether the synchronization performance remains stable or degrades over time, highlighting any drift or systematic changes in accuracy.

6. Results

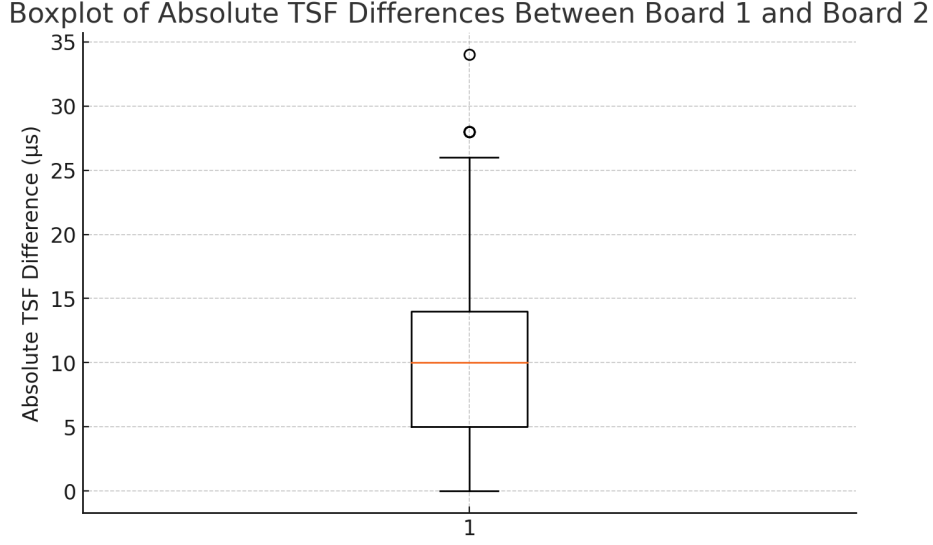


Figure 6.8.: Box plot of synchronization error between two clients, illustrating the distribution, variance, and precision of the timing alignment over the experiment duration.

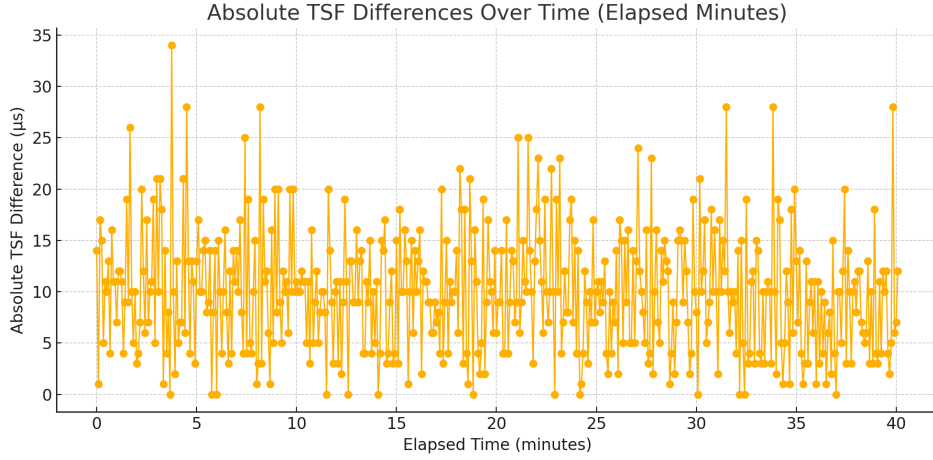


Figure 6.9.: Synchronization error between two clients as a function of time over a 40-minute period, used to assess long-term stability and potential drift in synchronization performance.

As shown by the results, the synchronization performance is excellent, with an average timing error of approximately 10 microseconds. Moreover, there is no significant drift observed over the 40-minute testing period, indicating stable and reliable synchronization throughout the experiment.

6. Results

6.2.2. TSF: Two Clients and AP

After evaluating synchronization between the two clients, the final step was to assess synchronization accuracy when including the access point (AP) itself. This is particularly relevant, as the IEEE 802.11 Time Synchronization Function (TSF) is designed to keep the timers of all stations including both clients and the AP tightly synchronized. Accurate synchronization with the AP is crucial for applications that require precise timing across the entire wireless network, such as coordinated sensing or distributed control.

To test this, we extended our experimental setup to monitor the TSF values of both clients and the AP during continuous data streaming. This allowed us to directly measure the synchronization error not only between the clients, but also between each client and the AP, providing a comprehensive view of timing alignment within the network.

The figure below presents the results of this experiment6.10:

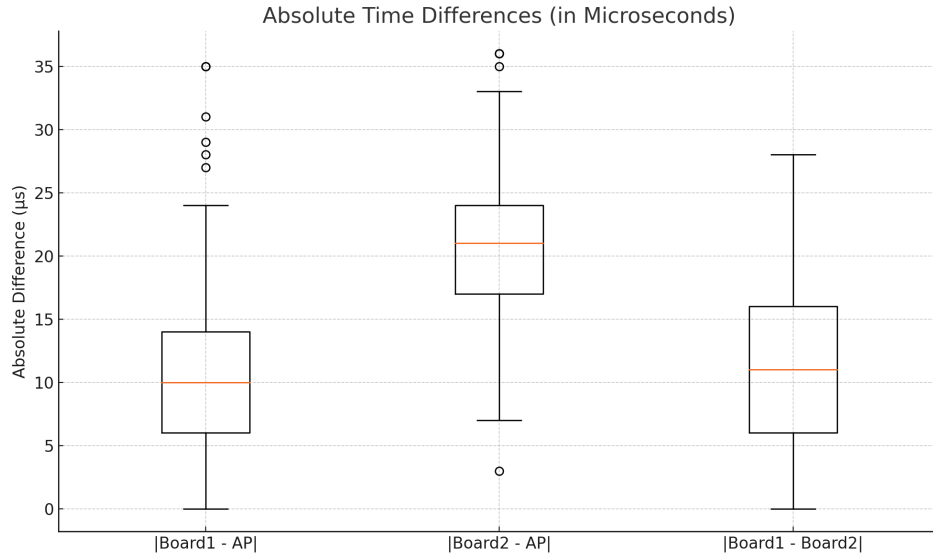


Figure 6.10.: Synchronization error between two clients and the AP over time, illustrating the TSF alignment across all nodes in the network.

As can be seen from the results, the synchronization remains excellent, not only between the two clients but also between each client and the access point. The timing alignment is consistently maintained across all nodes, further confirming the effectiveness of the TSF mechanism in providing reliable and precise synchronization within the network.

6. Results

6.2.3. Beacon Interval

With synchronization performance validated, the next step was to investigate whether increasing the beacon interval could further optimize power consumption without excessively degrading synchronization accuracy. By reducing the frequency of beacon transmissions, devices can spend more time in low-power states, potentially lowering overall energy usage.

To evaluate this trade-off, experiments were conducted with beacon intervals ranging from 100 ms up to 60 s. Both the absolute time difference between the client and AP, as well as the corresponding power consumption, were measured for each configuration.

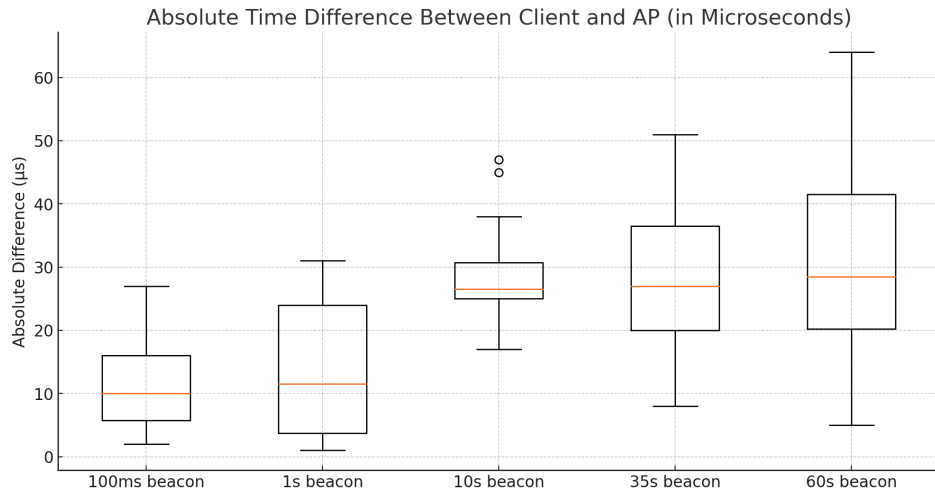


Figure 6.11.: Box plot of the absolute time difference between client and AP for various beacon intervals.

6. Results

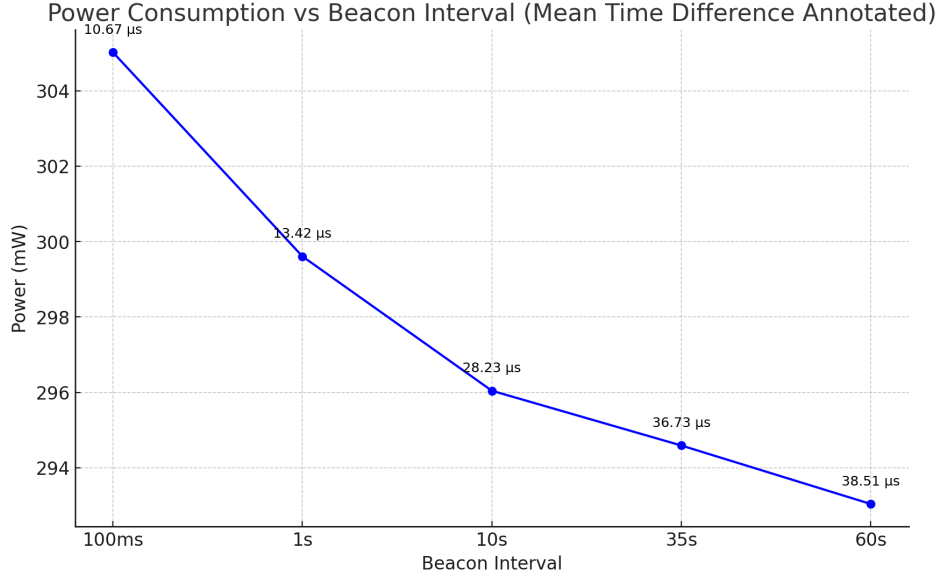


Figure 6.12.: Power consumption as a function of beacon interval. The mean synchronization error for each interval is annotated.

As shown in Figure 6.11, increasing the beacon interval does lead to a gradual increase in both the mean and variance of the absolute time difference between the client and the AP. However, even at the longest tested interval of 60 s, the average synchronization error remains below 40 microseconds, which is well within the acceptable range for most WBAN applications.

Figure 6.12 demonstrates that power consumption decreases steadily as the beacon interval increases. This confirms that less frequent beacons allow devices to remain in low-power states for longer periods, resulting in significant energy savings.

Overall, these results indicate that it is possible to achieve some power optimization by increasing the beacon interval, with only a modest impact on synchronization accuracy. This trade-off can be tuned to meet the specific requirements of the application, balancing energy efficiency and timing precision.

6.3. Extra

To complete the characterization of the system, additional experiments were conducted focusing on two key performance aspects: the impact of distance between the access point and the client, and the throughput behavior when two client boards are active simultaneously. These tests provide further insight into the robustness and limitations of the ESP32-based setup.

6. Results

The first experiment analyzed throughput when two boards are connected and transmitting concurrently. As shown in Figure 6.13, both devices exhibit comparable performance across increasing duty cycles. The maximum total throughput reached with two boards is approximately the same as that of a single board operating alone. This behavior suggests the presence of a bottleneck, likely caused by limitations in the ESP-based access point's ability to handle concurrent transmissions efficiently.

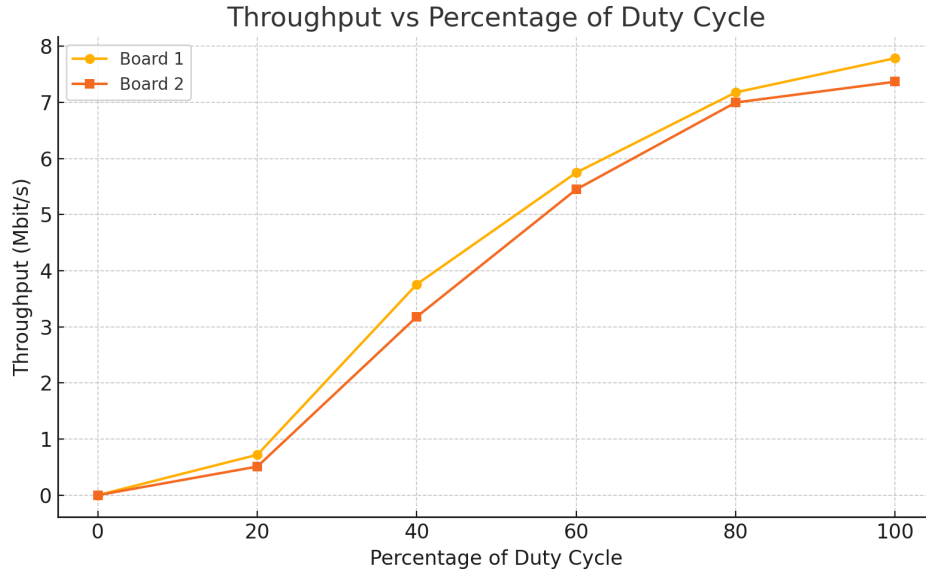


Figure 6.13.: Throughput comparison for two client boards under varying duty cycles.

The second experiment evaluated the throughput as a function of the distance from the access point. As expected, the throughput decreases as the distance increases, due to signal attenuation and potential interference. Figure 6.14 illustrates this trend for two different boards, showing a similar decay pattern for both devices.

6. Results

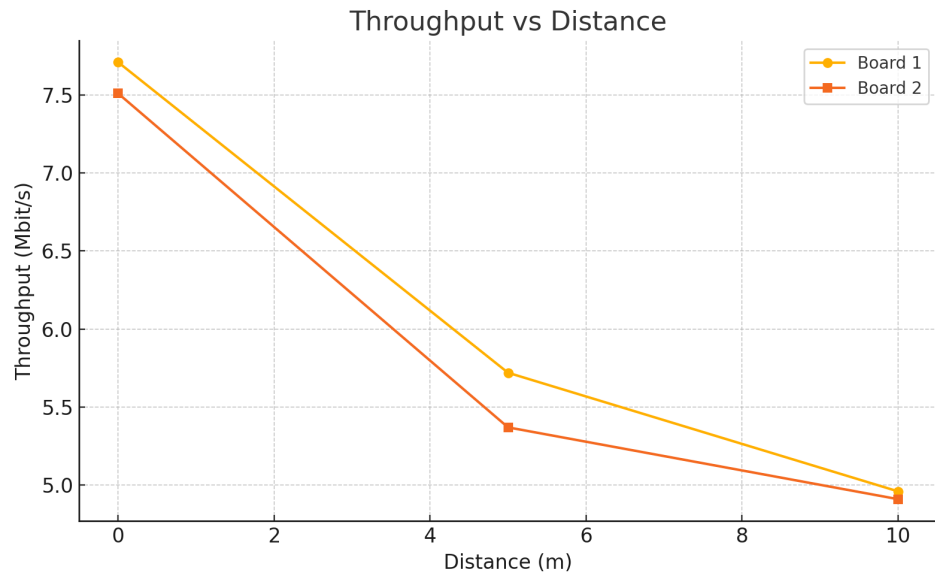


Figure 6.14.: Throughput degradation over increasing distance between client and access point.

Conclusion and Future Work

This thesis presented the design, implementation, and evaluation of an ultra-low power and high-throughput wireless network for continuous body monitoring, focusing on power optimization and time synchronization in Wireless Body Area Networks (WBANs). The main contributions include a detailed analysis of power consumption under various duty cycles, the integration of Target Wake Time (TWT) for energy efficiency, and the assessment of timing synchronization using the Timing Synchronization Function (TSF) within a custom firmware architecture.

7.1. Power Optimization

The investigation into power optimization strategies highlighted substantial improvements achievable with targeted approaches on the ESP32-C6 platform. Specifically, leveraging the ESP32-C6's modem-sleep and automatic light-sleep modes significantly reduced power consumption while maintaining consistent data throughput, essential for continuous and reliable Wireless Body Area Network (WBAN) operations.

Experimental evaluations demonstrated that modem-sleep mode provided a practical balance between power efficiency and system responsiveness, ideal for applications requiring persistent TCP connections. By selectively disabling the RF components during idle periods while keeping digital systems active, modem-sleep achieved substantial reductions in power usage without disrupting data integrity or system stability.

Automatic light-sleep further enhanced these energy savings by suspending nearly all SoC activities during longer idle intervals. This method allowed the system to dramatically lower idle currents to microampere levels. The integration of Target Wake Time (TWT), a novel feature enabled by Wi-Fi 6, allowed client nodes to negotiate precise wake-up

7. Conclusion and Future Work

schedules with Wi-Fi access points, further reducing power draw by avoiding unnecessary beacon checks and synchronizing wake periods more efficiently.

The experimental results illustrated clear benefits across different duty cycles and periods. Particularly notable was the flexibility provided by the duty cycle approach, which allowed precise tuning of power consumption based on the specific latency and throughput requirements of various biomedical monitoring scenarios. The combined utilization of automatic light-sleep, and TWT scheduling should a robust and adaptable system, capable of not only matching performance of other wireless technology but even improve them.

Overall, these power optimization techniques confirmed the ESP32-C6's strong potential for ultra-low-power WBAN deployments, significantly improving energy efficiency without compromising essential network performance and reliability.

7.2. Time Synchronization

The time synchronization tests assessed the effectiveness of the TSF mechanism across multiple network nodes. Results indicate that the implemented solution achieves low synchronization error and minimal drift over extended periods, even with simple and maintainable code. The synchronization accuracy is sufficient for most continuous body monitoring applications, ensuring reliable timestamp alignment across sensors and the access point. This is reflected in the low variance and high stability shown in the experimental results.

7.3. Summary and Future Directions

In summary, this thesis demonstrates that modern Wi-Fi, when enhanced with TWT and robust time synchronization, can serve as a viable platform for next-generation WBANs—offering both low power consumption and high throughput. Compared to related work, the proposed approach achieves competitive energy efficiency without sacrificing data rate, making it suitable for applications requiring continuous, high-bandwidth physiological data streaming.

Future work could focus on several promising directions:

- **Comprehensive TWT Evaluation:** Further analysis with more mature hardware and firmware support to fully exploit TWT's capabilities.
- **Scalability:** Extending the network to support a larger number of nodes and studying the impact on synchronization and energy consumption.
- **Testing with the BioGap platform:** Integrating the Biogap platform in to the already developed firmware

7. Conclusion and Future Work

- **Real-world Deployment:** Testing the system in real clinical or sports environments to validate its robustness and usability.

Overall, the results lay a strong foundation for the development of energy-efficient, high-performance WBANs, and open the door for further research into advanced features and optimizations.

Appendix	A
----------	----------

Task Description



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institut für Integrierte Systeme
Integrated Systems Laboratory

BACHELOR THESIS AT THE DEPARTMENT OF
INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING
SPRING SEMESTER 2025

**WI-Body: towards an ultra-low power and high
throughput network for continuous body monitoring**

22.02.2025

Student: Alberto Fasulo, afasulo@student.ethz.ch
Supervisors: Sebastian Frey (IIS), ETZ J 69.2, sefrey@iis.ee.ethz.ch
Giusy Spacone (IIS), ETZ J 68.2, gspacone@iis.ee.ethz.ch
Andrea Cossettini (IIS), OAT U27, cosandre@iis.ee.ethz.ch
Start: 03.03.2025
Due: 09.06.2025

The final report will be submitted in electronic format. All copies remain property of the Integrated Systems Laboratory.

1 Introduction

A wireless Body Area Network (WBAN) is a system of wireless, networked sensing devices worn or placed near the human body [1]. The concept has garnered significant interest in recent years due to its diverse applications, including mobile health monitoring, rehabilitation, fitness tracking, virtual and augmented reality, and human-machine interactions [1], [2]. Despite its potential, the development of an effective WBAN faces several technical challenges [2].

One key challenge is the selection of a suitable network topology, which is often dictated by the specific application. The topology impacts the efficiency and reliability of data transmission within the network. Another challenge involves choosing an appropriate communication protocol between the peripheral sensing nodes and the central node. Different protocols offer various advantages, such as extended communication range, enhanced security, or reduced power consumption. Effective energy management is crucial to ensure that the WBAN can operate for extended periods, ideally over multiple days, without frequent battery replacement or recharging. This is especially important for applications that require continuous data monitoring. The design of the sensing nodes, including sensor selection and node architecture, must also be optimized to meet the specific requirements of the application. Additionally, the computing paradigm must be chosen carefully, balancing the need for real-time data processing with power efficiency [1].

In this project, you will work on the development of a WBAN (Figure 1) with the following key features:

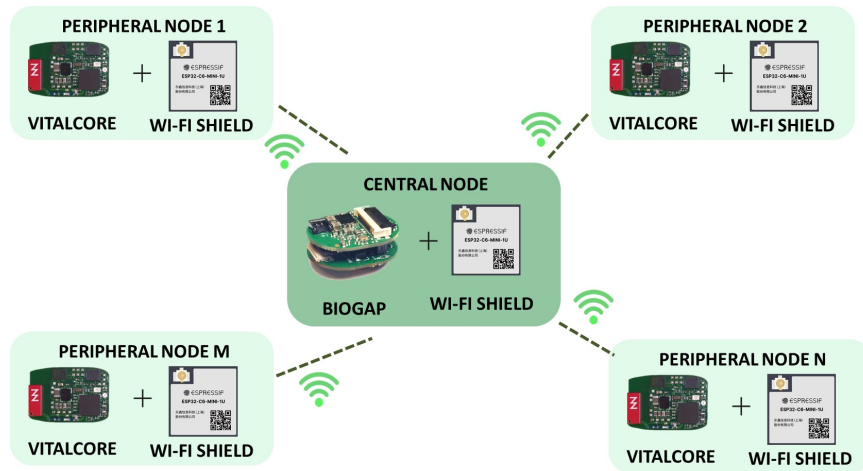


Figure 1: High-level overview of the Body Sensor Network (WBAN). The central node, built on the Biogap platform [3] and equipped with a custom shield for Wi-Fi connectivity, communicates with peripheral nodes based on the VitalCore [4] system. The central node manages data collection and processing from the peripheral nodes, ensuring secure and efficient wireless communication throughout the network.

- Star topology, where each sensing node communicates directly with a central node. The central is based on the Biogap [3] platform. Powered by the GAP9 [5] processor, the

central node will handle the data computation locally, eliminating the need for external devices.

- The sensing nodes are based on the VitalCore board [4], which is designed to monitor vital signals such as photoplethysmography (PPG), body temperature, and motion detection. In addition, custom shields can be integrated into the existing board to enable the monitoring of additional signals, such as biopotentials, making the network flexible and highly adaptable to a wide range of application requirements.
- Wi-Fi communication protocol, chosen for its high bandwidth capabilities, which are necessary to stream large volumes of data, especially envisioning sensing modalities like ultrasound. The ESP32-C6-MINI-1 [6] will be employed to facilitate this communication. Crucially, raw data will be streamed from each sensing node to the central node, enabling sensor fusion and energy-efficient edge computing.

2 Project Goals

The main tasks of this project are:

- **Task 1 - Literature review and familiarization with project specifics**
 1. Review of existing WBAN architectures and communication protocols.
 2. Review of WBANs designed for monitoring human body activity, for health, wellness, or fitness purposes.
 3. Familiarization with the platforms and previous work for the project.
- **Task 2 - Bring-up of Wi-Fi Shield**
 1. Assemble and integrate the previously designed Wi-Fi shield.
 2. Validate the hardware functionality, ensuring all components work correctly.
 3. Flash firmware onto the ESP32 SoC on the Wi-Fi shield and conduct functional tests (basic data streaming, throughput measurement) based on previously developed code.
- **Task 3 - Firmware Development for ESP32-C6**
 1. Implement data streaming between sensing nodes and the central node.
 2. Optimize for energy efficiency and throughput using Wi-Fi 6 features (e.g., Target Wake Time).
 3. Implement a mechanism for time synchronization across different nodes.
- **Task 4 - Firmware Development for nRF5340 on BioGAP Mainboard**
 1. Establish a communication protocol between the BioGAP Mainboard and the ESP32-C6 on the Wi-Fi shield.
 2. Integrate sensor acquisition drivers into the project (e.g., ExG, PPG, IMU, microphone).
 3. Ensure efficient and reliable data transfer between components.

- **Task 5 - System Characterization**

1. Measure power consumption and throughput.
2. Characterize the time synchronization accuracy between nodes.
3. (Optional) Validate WBAN performance with functional, multimodal data acquisition.

- **Task 5 - Gather and Present Final Results**

1. Gather final results.
2. Prepare presentation (15 min. + 5 min. discussion).
3. Write a final report.

3 Project Realization

3.1 Project Plan

Within the first month of the project you will be asked to prepare a project plan. This plan should identify the tasks to be performed during the project and sets deadlines for those tasks. The prepared plan will be a topic of discussion of the first week's meeting between you and your advisers. Note that the project plan should be updated constantly depending on the project's status.

3.2 Meetings

Weekly meetings will be held between the student and the assistants. The exact time and location of these meetings will be determined within the first week of the project in order to fit the students and the assistants schedule. These meetings will be used to evaluate the status and progress of the project. Beside these regular meetings, additional meetings can be organized to address urgent issues as well.

3.3 Report

Documentation is an important and often overlooked aspect of engineering. One final report has to be completed within this project. The common language of engineering is de facto English. Therefore, the final report of the work is preferred to be written in English. Any form of word processing software is allowed for writing the reports, nevertheless the use of L^AT_EX with Tgif¹ or any other vector drawing software (for block diagrams) is strongly encouraged by the IIS staff.

Final Report The final report has to be presented at the end of the project and a digital copy need to be handed in. Note that this task description is part of your report and has to be attached to your final report.

¹See: <http://bourbon.usc.edu:8001/tgif/index.html> and <http://www.dz.ee.ethz.ch/en/information/how-to/drawing-schematics.html>.

3.4 Presentation

There will be a presentation (15 min presentation and 5 min Q&A) at the end of this project in order to present your results to a wider audience. The exact date will be determined towards the end of the work.

References

- [1] R. Gravina and G. Fortino, “Computational aspects in body sensor networks (BSNs): From operating systems to data fusion,” in *Encyclopedia of Sensors and Biosensors (First Edition)* (R. Narayan, ed.), pp. 173–185, Elsevier.
- [2] D. M. G. Preethichandra, L. Piyathilaka, U. Izhar, R. Samarasinghe, and L. C. De Silva, “Wireless body area networks and their applications—a review,” vol. 11, pp. 9202–9220.
- [3] S. Frey, M. Guermandi, S. Benatti, V. Kartsch, A. Cossettini, and L. Benini, “BioGAP: a 10-core FP-capable ultra-low power IoT processor, with medical-grade AFE and BLE connectivity for wearable biosignal processing.”
- [4] P. Schilk, K. Dheman, and M. Magno, “Vitalpod: A low power in-ear vital parameter monitoring system,” in *2022 18th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)*, pp. 94–99, 2022.
- [5] “GAP9 Manuals.” <https://greenwaves-technologies.com/gap9-resources/>. Accessed: 18-07-2024.
- [6] E. Systems, “Esp32 wi-fi bluetooth soc.” https://www.espressif.com/sites/default/files/documentation/esp32-c6-mini-1_mini-1u_datasheet_en.pdf.

Zürich, October 11, 2024

Prof. Dr. Luca Benini

Appendix	B
----------	----------

Declaration of Originality



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

- ☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☒ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

WI-BODY: TOWARDS AN ULTRA-LOW POWER AND HIGH THROUGHPUT
NETWORK FOR CONTINUOUS BODY MONITORING

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

FASULO

First name(s):

ALBERTO

With my signature I confirm the following:

- I have adhered to the rules set out in the [Citation Guidelines](#).
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

ZÜRICH 09.06.2025

Signature(s)

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ For further information please consult the ETH Zurich websites, e.g. <https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html> and <https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html> (subject to change).

Bibliography

- [1] M. Salayma, A. Al-Dubai, I. Romdhani, and Y. Nasser, “Wireless body area network (wban): A survey on reliability, fault tolerance, and technologies coexistence,” in *Proceedings of the 2017 International Conference on Future Networks and Distributed Systems (ICFNDS '17)*. New York, NY, USA: Association for Computing Machinery, 2017, pp. 1–9. [Online]. Available: <https://dl.acm.org/doi/10.1145/3041956>
- [2] E. Systems, “Espressif announces the esp32-c6: World’s first risc-v wi-fi 6 + bluetooth 5 (le) soc,” https://www.espressif.com/en/news/ESP32_C6, 2023, accessed 15 May 2025.
- [3] “Keysight e3647a,” <https://us.rs-online.com/product/keysight-technologies/e3647a/70180142/>.
- [4] “Keysight 33210a,” <https://www.keysight.com/us/en/product/33210A/33210a-waveform-and-function-generator.html>.
- [5] “Esp32-c6 image,” <https://www.vroque.co/post/conociendo-el-nuevo-procesador-esp32-c6-de-espressif>.
- [6] “Breadboard image,” <https://openclipart.org/detail/216110/breadboard>.
- [7] A. Mahmood, M. Iqbal, I. Ahmed, A. Anwar, and D. Harada, “Low-power wide-area networks: A survey of enabling technologies, applications and interoperability needs,” *Wireless Networks*, vol. 27, no. 8, pp. 5271–5295, 2021. [Online]. Available: <https://doi.org/10.1007/s11276-021-02780-2>
- [8] M. Bajcsy, M. Cerny, P. Dohnalkova, R. Martinek, V. Roubicek, and O. Svarny, “Performance analysis for cycling with ieee 802.15.4,” *Personal and Ubiquitous Computing*, vol. 15, no. 7, pp. 645–654, 2011. [Online]. Available: <https://link.springer.com/article/10.1007/s00779-011-0486-x>

Bibliography

- [9] A. Pantelopoulos and N. G. Bourbakis, “A survey on wearable sensor-based systems for health monitoring and prognosis,” *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 40, no. 1, pp. 1–12, 2010. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/4201256>
- [10] A. Alomainy, N. Nasr, T. Bhatti, and N. Qaddoumi, “Real time patient monitoring system using zigbee wireless sensor network,” *Procedia Computer Science*, vol. 65, pp. 1164–1173, 2015. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050915032196>
- [11] S.-W. Chen, Y. Xue, W. Li, and H.-B. He, “Real-time eeg monitoring based on wireless sensor network,” *Journal of Medical Systems*, vol. 33, pp. 373–381, 2009. [Online]. Available: <https://link.springer.com/article/10.1007/s10916-008-9242-9>
- [12] Y.-C. Liu, C.-H. Chang, Y.-T. Chen, W.-C. Hong, F.-R. Su, and J.-J. Lai, “A zigbee-based mobile healthcare system,” in *2009 Sixth International Conference on Networked Computing and Advanced Information Management*. IEEE, 2009, pp. 620–625. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4649394>
- [13] Y. Yu, X. Du, M. Zhou *et al.*, “Low-power design of wireless body area network,” in *2010 International Conference on Information Science and Engineering (ICISE)*. IEEE, 2010, pp. 448–451. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=5208087>
- [14] H.-L. Xu, X.-Y. Zhu, J.-Q. Wang, and X.-N. Yang, “Design of a wireless body area network for health monitoring based on bluetooth,” in *2011 International Conference on Electronic and Mechanical Engineering and Information Technology*, vol. 1. IEEE, 2011, pp. 381–384. [Online]. Available: <https://ieeexplore.ieee.org/document/6098710>
- [15] F. Touati, R. Tabish, and D. Benhaddou, “An experimental performance evaluation and compatibility study of the bluetooth low energy based platform for ecg monitoring in wbans,” *Procedia Computer Science*, vol. 37, pp. 399–406, 2014. [Online]. Available: <https://pdf.sciencedirectassets.com/282166/1-s2.0-S2212670813X0003X/1-s2.0-S2212670813001243/main.pdf>
- [16] D. Cavalcanti *et al.*, “Performance analysis of wi-fi 6 for wireless body area networks in healthcare applications,” *Electronics*, vol. 12, no. 20, p. 4226, 2023. [Online]. Available: <https://www.mdpi.com/2079-9292/12/20/4226>
- [17] J. Smith *et al.*, “A review of wi-fi technologies for future wireless body area networks,” *Future Internet*, vol. 16, no. 2, p. 36, 2024. [Online]. Available: <https://www.mdpi.com/1999-5903/16/2/36>

Bibliography

- [18] J. Caredda, “Wi-body: Towards an ultra-low power and high throughput network for continuous body monitoring,” Zurich, Switzerland, 2025. [Online]. Available: https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/63487887/ffc6baf4-784e-49ea-9c35-7dff6f4bef59/report_WiFi_shield_WBAN.pdf
- [19] S. Frey, M. Guermendi, S. Benatti, V. Kartsch, A. Cossettini, and L. Benini, “Biogap: a 10-core fp-capable ultra-low power iot processor, with medical-grade afe and ble connectivity for wearable biosignal processing,” in *2023 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*. IEEE, 2023, pp. 1–7. [Online]. Available: <https://www.research-collection.ethz.ch/bitstream/handle/20.500.11850/646588/BioGAP.pdf?sequence=1&isAllowed=y>
- [20] M. U. Khan *et al.*, “A survey on wireless wearable body area networks,” *Sensors*, vol. 22, no. 20, p. 7722, 2022. [Online]. Available: <https://www.mdpi.com/1424-8220/22/20/7722>
- [21] G. Sharma *et al.*, “The state-of-the-art wireless body area sensor networks: A survey,” *International Journal of Distributed Sensor Networks*, vol. 14, no. 7, p. 1550147718768994, 2018. [Online]. Available: <https://journals.sagepub.com/doi/full/10.1177/1550147718768994>
- [22] Y. Zhang *et al.*, “Gateway over the air: towards pervasive internet connectivity for commodity iot devices,” in *Proceedings of the 18th ACM Conference on Embedded Networked Sensor Systems (SenSys)*, pp. 331–344.
- [23] E. Mozaffariahrar, M. Menth, and S. Avallone, “Implementation and evaluation of ieee 802.11ax target wake time feature in ns-3,” *GetMobile: Mobile Computing and Communications*, vol. 28, no. 1, pp. 27–33, 2024. [Online]. Available: <https://dl.acm.org/doi/fullHtml/10.1145/3659111.3659112>